

Todos los algoritmos de la materia

Inteligencia Computacional · FICH-UNL · cada ecuación en orden, y debajo qué dice en palabras

Los resúmenes de cada unidad explican **por qué** funciona cada pieza, con las derivaciones y los ejemplos numéricos. Este archivo es sólo el **qué**: los algoritmos, uno atrás del otro, en el orden en que se ejecutan. Cada ecuación con un párrafo debajo que dice qué significa. Sirve para repasar de corrido y para desarrollar en la pizarra.

#	Algoritmo	Supervisado	Entrenamiento
1	Perceptrón simple — corrección de error	sí	iterativo
2	Perceptrón simple — gradiente (LMS)	sí	iterativo
3	Perceptrón multicapa — back-propagation	sí	iterativo
4	Base radial	mixto	dos fases
5	Hopfield	no	directo
6	BPTT	sí	iterativo
7	SOM	no	iterativo
8	LVQ1	sí	iterativo
9	Capacidad de generalización y evaluación	—	procedimiento

Cómo está armada cada sección. Arriba, el recuadro violeta «contame sobre...»: la respuesta de un minuto para cuando te piden que desarrolles el tema, en el orden *qué es* → *cómo está armado* → *cómo aprende* → *qué lo distingue* → *su límite*. Después la idea de fondo, las ecuaciones en orden, y al final la respuesta de treinta segundos con las repreguntas.

Convenciones que cambian entre algoritmos — las que más se mezclan en la pizarra:

- **Umbral:** entra como un peso más, w_0 , con entrada fija $x_0 = -1$.
- **Signo del error:** perceptrón y multicapa usan $e = d - y$ y la regla **suma**; RBF (fase 2) y BPTT usan $e = y - d$ y la regla **resta**. Es la misma cuenta escrita al revés.
- **Paso:** η en la corrección de error, μ en LMS y back-propagation (la equivalencia entre ellos se indica en cada sección), α en LVQ.
- **Derivada de la sigmoide simétrica** siempre en función de la salida: $\varphi'(v) = \frac{1}{2}(1 + y)(1 - y)$.

1. Perceptrón simple — corrección de error

SI TE DICEN — «contame sobre el perceptrón simple»

«Es el modelo más simple de neurona artificial. Hace el producto interno entre sus pesos y la entrada —con el umbral metido como un peso más, con entrada fija -1 — y le aplica la función signo: la salida es ± 1 , así que clasifica en dos clases. Geométricamente traza un **hiperplano** y dice de qué lado cayó cada punto. Se entrena con supervisión, patrón por patrón: si acierta no se toca nada; si se equivoca, a los pesos se les suma $\frac{\eta}{2}$ por el error por la entrada, y eso **gira el hiperplano** hacia el lado correcto. Se repite hasta una pasada completa sin errores. Y el límite es la clave: eso está garantizado **sólo si el problema es linealmente separable**; con el XOR no converge nunca, y eso es lo que motiva el multicapa.»

IDEA DE FONDO — en qué consiste

Una sola neurona que traza una **recta** (un hiperplano) y dice de qué lado cayó cada punto. Tiene dos partes que se repiten patrón por patrón: **calcular la salida** y, **sólo si se equivocó, mover la recta**. La imagen para recordarlo: la recta está mal puesta, entra un punto que quedó del lado equivocado, y la recta **gira un poco hacia ese punto**. Si el punto ya estaba bien, no pasa nada. Repetido muchas veces, la recta se acomoda hasta no equivocarse con ninguno — y eso está garantizado **sólo si los datos se pueden separar con una recta**.

Datos de partida

$$\{(\mathbf{x}(n), d(n))\}, \quad x_0 = -1$$

Pares entrada-salida deseada. La componente $x_0 = -1$ es fija y existe para que el umbral entre como un peso más, $w_0 = u$, y no haya que tratarlo aparte.

Salida

$$y = \varphi(\langle \mathbf{w}, \mathbf{x} \rangle)$$

Se hace el producto interno entre los pesos y la entrada, y se le aplica la función de activación. Todo el perceptrón es eso.

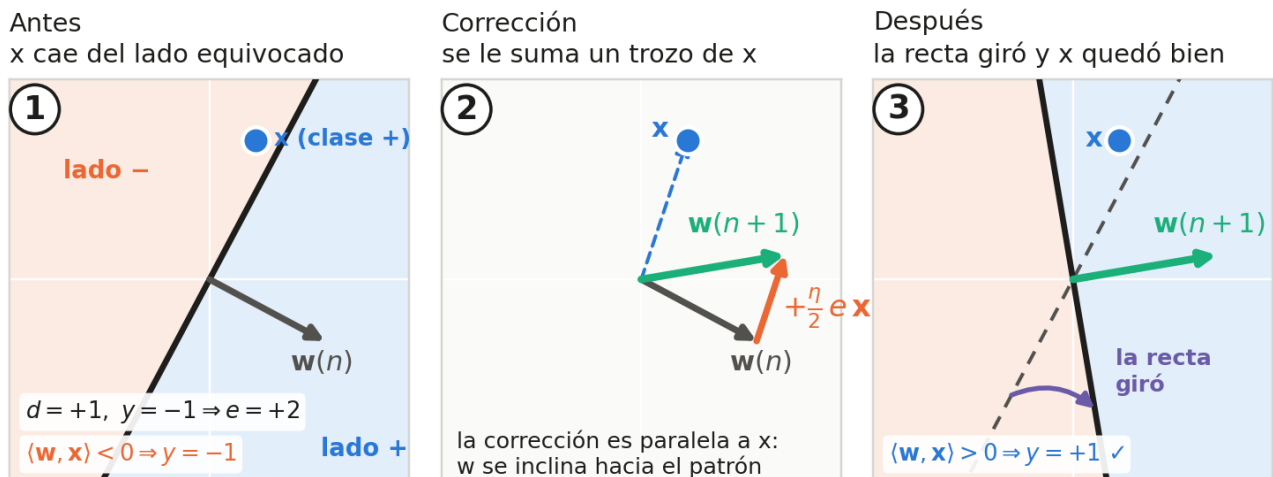
$$\langle \mathbf{w}, \mathbf{x} \rangle = 0$$

Los puntos donde el producto interno da cero son la **frontera de decisión**: un hiperplano que parte el espacio de entrada en dos semiplanos.

Adaptación

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\eta}{2} [d(n) - y(n)] \mathbf{x}(n)$$

Si la salida coincide con la deseada, el corchete es cero y los pesos no se tocan. Si no coincide, los pesos se mueven en la dirección de la entrada, con el signo del error: la frontera gira hacia el lado que corrige ese patrón.



Si x ya estaba del lado correcto: $e = 0 \rightarrow$ no se toca nada. (Umbral omitido para poder dibujar en 2D.)

Figura 1: El producto interno decide el lado; la corrección inclina $\mathbf{w}$ hacia el patrón y la recta, que es perpendicular, gira con él.

Parada

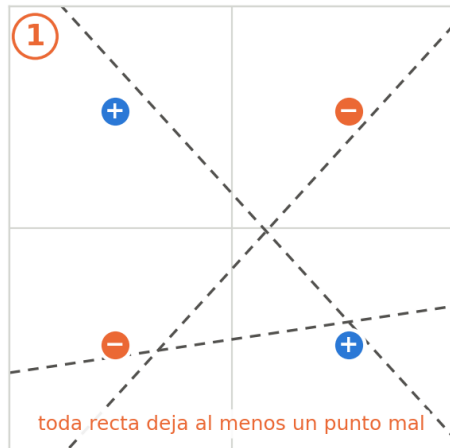
$$d(n) = y(n) \quad \forall n$$

Se termina cuando una pasada completa por el conjunto no produce ninguna corrección. Está garantizado que eso pasa **sólo si el problema es linealmente separable**.

Cómo se lee la regla

$e(n)$ es un **escalar** y $\mathbf{x}(n)$ un **vector**, así que la corrección $\frac{\eta}{2}e(n)\mathbf{x}(n)$ es un vector **en la dirección del patrón de entrada**, con magnitud proporcional al error y signo según si la salida se quedó corta o larga. Cada paso empuja a $\mathbf{w}$ a lo largo de $\mathbf{x}$, y como $\mathbf{w}$ es la normal de la frontera, la recta **gira** hacia el lado que corrige ese patrón.

Con una recta, no
el perceptrón simple oscila para siempre



Con dos rectas, sí
una capa oculta de 2 neuronas: la franja

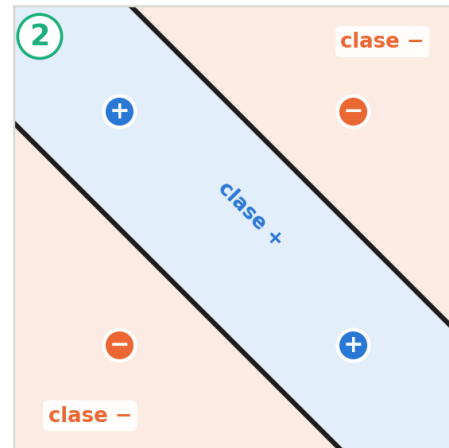


Figura 2: Con XOR no existe recta: la frontera oscila para siempre. Con dos rectas —una capa oculta— sí se separa.

PARA LA DEFENSA — la respuesta de treinta segundos

«Una neurona que traza un hiperplano y dice de qué lado cayó cada punto. Se le muestra un patrón: si acierta no se toca nada, y si se equivoca los pesos se mueven en la dirección del patrón, con el signo del error. Eso gira la frontera hacia el lado correcto. Se repite hasta una pasada completa sin correcciones, y eso está garantizado sólo si los datos son linealmente separables.» **La repregunta segura:** ¿y si no son separables? No converge: queda oscilando. Ése es el límite que motiva el multicapa, y el ejemplo es el XOR.

2. Perceptrón simple — regla del gradiente (LMS)

SI TE DICEN — «contame sobre LMS»

«Es la misma neurona con otra lógica de entrenamiento. En vez de corregir sólo cuando se equivoca, defino una función que **mide el error** —el error cuadrático— y muevo los pesos **en contra de su gradiente**, que es la dirección de máximo descenso. Para poder derivar uso la neurona **lineal**, sin el signo, porque el signo no es derivable. La cuenta da la regla de Widrow-Hoff: $\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e \mathbf{x}$, **velocidad por error por entrada**. El punto delicado es μ : grande, oscila y se queda con el último patrón; chico, converge pero muy lento. Lo importante es que fija **el esquema de todo lo supervisado que viene después**: multicapa, radial y BPTT también bajan por el gradiente, sólo que con un gradiente más difícil de calcular.»

IDEA DE FONDO — en qué consiste

La misma neurona, pero con otra filosofía: en vez de reaccionar sólo cuando se equivoca, **define un número que mide lo mal que está** y se mueve siempre en la dirección que más lo baja. Tres partes: **definir el criterio de error**, **derivarlo** respecto de los pesos, y **moverse en contra de esa derivada**. La imagen: una pelota bajando por una ladera, donde la altura es el error y la posición son los pesos. Es el molde de todos los algoritmos supervisados que vienen después — multicapa, radial y BPTT son este mismo esquema con un gradiente más difícil de calcular.

Mismo modelo de neurona; cambia de dónde sale el ajuste. Acá no se corrige el error, se minimiza una función.

Criterio

$$e^2(n) = [d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2, \quad e(n) = d(n) - y(n)$$

Se define una medida de lo mal que está la red para el patrón actual: el error al cuadrado. Se trabaja en el **caso lineal** — $y = \langle \mathbf{w}, \mathbf{x} \rangle$, sin activación— porque sgn no es derivable. Y el criterio va **sin** el $\frac{1}{2}$: por eso el resultado lleva un 2μ .

Regla del gradiente

$$\Delta \mathbf{w}(n) = -\mu \nabla_{\mathbf{w}} e^2(n)$$

Los pesos se mueven en la dirección **opuesta** al gradiente, que es la de máximo crecimiento del error: se baja por la superficie de error. μ dice qué tan grande es el paso.

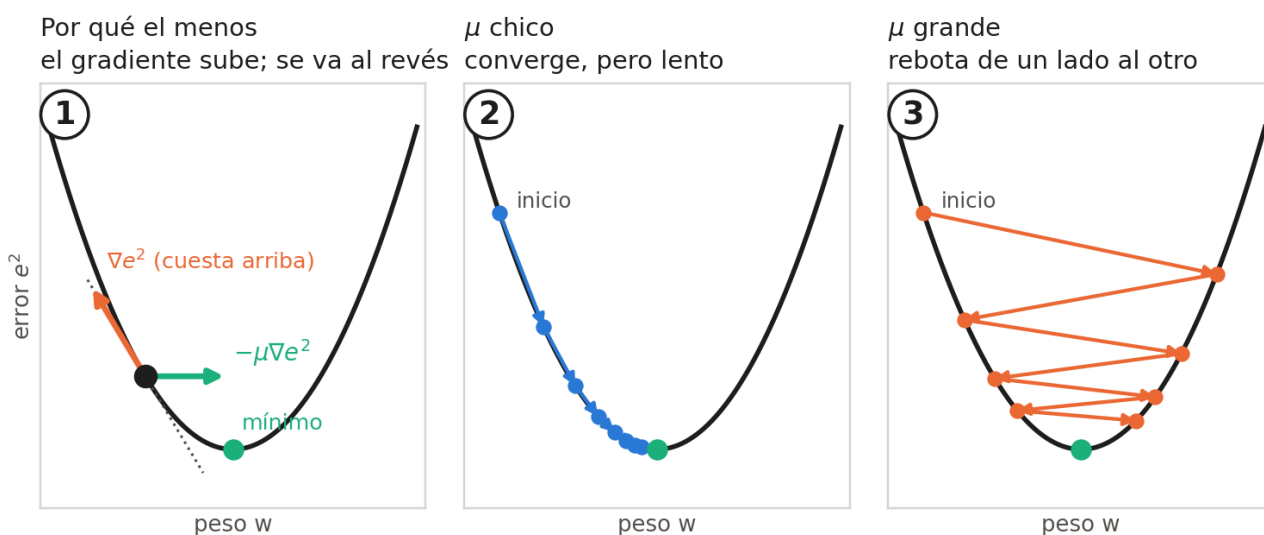


Figura 3: El gradiente apunta cuesta arriba; por eso el menos. Y el tamaño del paso decide si baja o rebota.

LMS (Widrow-Hoff)

$$\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n) \mathbf{x}(n)$$

Resolviendo el gradiente para este criterio queda esto: paso, por error, por entrada. Es la misma forma que la

corrección de error, y de hecho coinciden con $\eta = 4\mu$.

Cómo se lee la regla

La estructura **velocidad de aprendizaje** $\times$ **error** $\times$ **entrada** es la misma de la sección anterior, pero el origen es distinto: no sale de reaccionar ante un error, sale de **derivar una función y moverse en contra del gradiente**. Y ésa es la forma que se va a repetir en el multicapa, en la fase 2 de las radiales y en BPTT — lo único que cambia en cada uno es **cuán difícil es calcular ese gradiente**.

PARA LA DEFENSA — la respuesta de treinta segundos

«Misma neurona, otra filosofía: en vez de corregir cuando se equivoca, defino un número que mide lo mal que está —el error cuadrático— y me muevo siempre en la dirección que más lo baja, que es la opuesta al gradiente. La cuenta se hace en el caso lineal, sin activación, porque el signo no es derivable. El resultado es paso por error por entrada, y coincide con la corrección de error salvo una constante.» **Las dos repreguntas seguras:** por qué el menos (el gradiente apunta cuesta **arriba**, al máximo crecimiento del error) y qué pasa con μ (grande: oscila y la red se aprende el último patrón y se olvida los anteriores; chico: converge pero lentísimo).

Acá aparece por primera vez la estructura que se repite en todo lo que sigue: **velocidad de aprendizaje, por error, por entrada**.

3. Perceptrón multicapa — back-propagation

SI TE DICEN — «contame sobre el perceptrón multicapa»

«Agrega **capas ocultas** con activación sigmoidea —la simétrica, entre -1 y 1 —. Combinando semiplanos puede formar regiones arbitrarias, así que ya no depende de que el problema sea lineal: resuelve el XOR. Se entrena con **retropropagación**, que es descenso por gradiente del error cuadrático, en tres pasos por patrón: **hacia adelante** capa por capa, **error** en la salida, y **deltas hacia atrás**. El problema central es que **las ocultas no tienen salida deseada**. Se resuelve con el **delta**, el gradiente de error local: en la salida es error por derivada de la activación; en una oculta se arma con los deltas de la capa siguiente, pesados por las conexiones que las unen. Así todos los pesos se ajustan con la misma regla: μ **por delta por entrada**. Los riesgos: mínimos locales, saturación (derivada casi cero) y sobreajuste, que se controla con un conjunto de monitoreo.»

IDEA DE FONDO — en qué consiste

Varias capas de neuronas, cada una alimentando a la siguiente. Por cada patrón se hacen **tres cosas en este orden**: **ir hacia adelante** calculando las salidas de capa en capa, **medir el error** en la última, y **volver hacia atrás repartiendo la culpa**. El problema que resuelve, y es todo el tema: una neurona oculta **no tiene salida deseada**, así que no se sabe cuánto se equivocó. La solución: su error se arma **juntando los errores de todas las neuronas que ella alimenta**, cada uno pesado por la conexión que las une. Eso es lo que significa «retropropagar»: el error viaja hacia atrás por los mismos cables que la señal viajó hacia adelante.

Arquitectura

$$\mathbf{v}^{(p)} = \mathbf{W}^{(p)} \mathbf{y}^{(p-1)} \quad y_j^{(p)} = \varphi(v_j^{(p)})$$

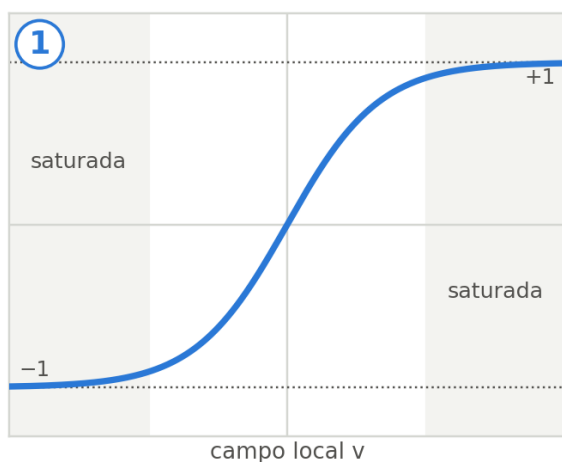
Cada capa toma la salida de la anterior, la multiplica por su matriz de pesos y le aplica la activación. $\mathbf{W}^{(p)}$ es de $M_p \times (M_{p-1} + 1)$: la columna extra es la del sesgo.

$$\varphi(v) = \frac{2}{1 + e^{-bv}} - 1$$

La sigmoide **simétrica**, que va de -1 a $+1$. Se usa ésta y no la logística porque las salidas centradas en cero hacen que el aprendizaje sea más parejo.

$$\varphi(v) = \frac{2}{1 + e^{-v}} - 1$$

simétrica: salidas centradas en 0



$$\varphi'(v) = \frac{1}{2}(1 + y)(1 - y)$$

se calcula con la salida y, que ya está

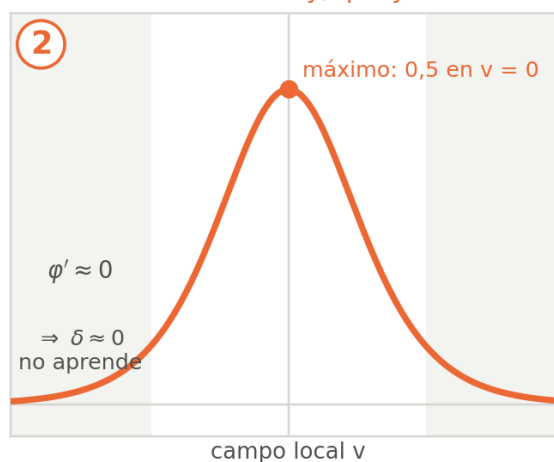


Figura 4: La derivada se apaga en los extremos: neurona saturada, $\delta \approx 0$, parálisis.

Error

$$e_j(n) = d_j(n) - y_j(n) \quad \xi(n) = \frac{1}{2} \sum_{j=1}^M e_j^2(n)$$

El error de cada neurona de salida, y el error instantáneo de la red como la suma de todos al cuadrado.

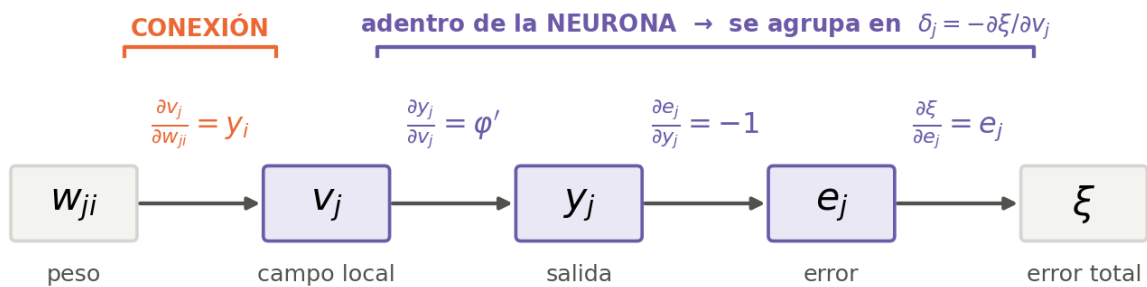
Regla del gradiente

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

Lo mismo que en el perceptrón, pero ahora el peso puede estar en cualquier capa y no hay salida deseada para las ocultas. Eso es lo que hay que resolver.

$$\frac{\partial \xi}{\partial w_{ji}} = \frac{\partial \xi}{\partial e_j} \frac{\partial e_j}{\partial y_j} \frac{\partial y_j}{\partial v_j} \frac{\partial v_j}{\partial w_{ji}}$$

La cadena a derivar. Los dos primeros factores dan el error, el tercero es la derivada de la activación y el cuarto es la entrada que llega por esa conexión.



$$\frac{\partial \xi}{\partial w_{ji}} = -\delta_j y_i \implies \Delta w_{ji} = \mu \delta_j y_i$$

el δ se calcula UNA vez por neurona y sirve para todos los pesos que le entran

Figura 5: El corte del δ : agrupa los tres factores de adentro de la neurona y deja afuera el único que depende de la conexión.

La derivada de la activación

$$\varphi'(v_j) = \frac{1}{2}(1 + y_j)(1 - y_j)$$

Se escribe en función de la **salida**, no del campo local: como y_j ya está calculado del paso hacia adelante, la derivada sale gratis. El $\frac{1}{2}$ va (con b general sería $b/2$).

Los deltas

$$\delta_j(n) = -\frac{\partial \xi}{\partial y_j} \varphi'(v_j)$$

Se le pone nombre a todo lo que pasa **dentro** de la neurona, para separarlo de lo que pasa en la conexión.

$$\delta_j = -\frac{\partial \xi}{\partial y_j} \frac{\partial y_j}{\partial v_j} = -\frac{\partial \xi}{\partial v_j}$$

Los dos factores de la definición son la regla de la cadena de **una sola** derivada: la del error respecto del **campo local**. Así se lee el δ en una línea: **cuánta culpa tiene esa neurona en el error total** — cuánto bajaría el error si su estímulo neto cambiara un poquito, y en qué sentido conviene moverlo.

Su nombre completo es **gradiente de error local instantáneo**, y cada palabra está justificada: **gradiente** porque es una derivada, **local** porque es de esa neurona en particular, **instantáneo** porque está evaluado en la iteración n .

Y el corte no es arbitrario: de los cuatro factores de la cadena, el δ agrupa **los tres que dependen de la neurona** y deja afuera **el que depende de la conexión** (y_i). Por eso se calcula **una sola vez por neurona** y se reutiliza

para todos los pesos que entran a ella — y por eso es **lo único que una capa necesita de la capa siguiente**, que es lo que hace posible propagar hacia atrás.

PARA LA DEFENSA — por qué la definición no se escribe con e_j , y el signo

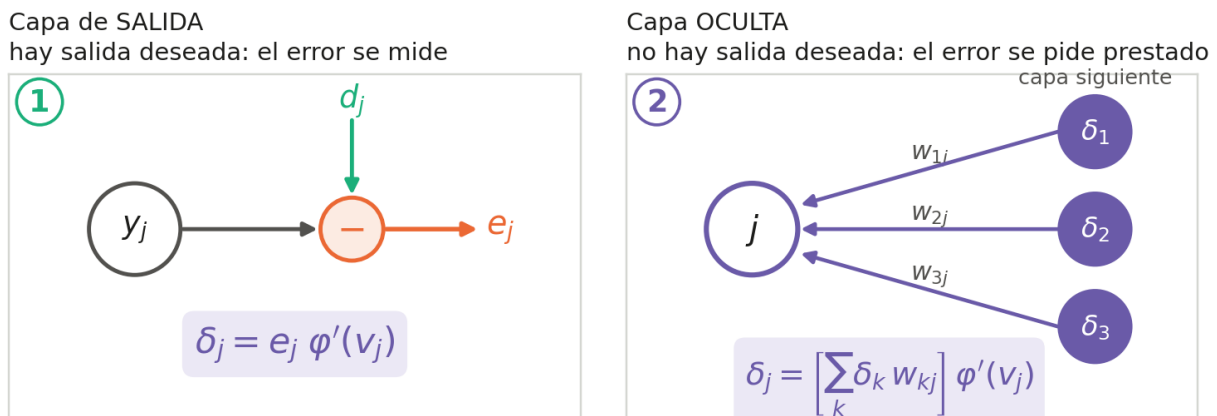
Porque **una neurona oculta no tiene e_j** : no hay salida deseada para ella. Lo que sí existe siempre es $\partial \xi / \partial y_j$, que vale $-e_j$ en la capa de salida y $-\sum_k \delta_k w_{kj}$ en una oculta. La definición genérica es la única que sirve para los dos casos; definirla como $e_j \varphi'$ te deja sin forma de bajar a las ocultas. **El menos** de la definición es el que hace que la actualización quede **sumando**: se come el $-\mu$ de la regla del gradiente. En clase el δ aparece primero sin el menos y enseguida con él; la versión que se usa es **con** el menos. **La respuesta de treinta segundos**: «El delta es el gradiente de error local instantáneo: menos la derivada del error respecto del campo local de esa neurona. Agrupa todo lo que pasa adentro de la neurona y deja afuera lo de la conexión, y por eso el ajuste queda delta por entrada. En la capa de salida sale del error verdadero; en una oculta, de los deltas de la capa siguiente pesados por los pesos que las unen.»

$$\delta_j^{III} = \frac{1}{2} e_j (1 + y_j^{III})(1 - y_j^{III})$$

En la capa de salida hay salida deseada, así que el delta es directo: error por derivada de la activación.

$$\delta_j^{II} = \left[\sum_k \delta_k^{III} w_{kj}^{III} \right] \frac{1}{2} (1 + y_j^{II})(1 - y_j^{II})$$

En una capa oculta no hay salida deseada, así que el error se trae de atrás: se juntan los deltas de la capa siguiente, cada uno pesado por la conexión que lo une con esta neurona. **El corchete reemplaza al error**, y el resto de la fórmula es idéntico.



Lo único que cambia entre las dos: el corchete ocupa el lugar de e_j . El resto de la fórmula es idéntico.

Figura 6: Arriba el error se mide; abajo se pide prestado a las neuronas de adelante, pesado por los pesos que las unen.

Actualización

$$\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n) \quad w_{ji}(n+1) = w_{ji}(n) + \Delta w_{ji}(n)$$

La regla vale para cualquier capa: paso, por delta de la neurona a la que llega la conexión, por la señal que entra por ella. Lo único que cambia entre capas es cómo se calculó el delta.

$$\Delta w_{j0}^{(p)} = \mu \delta_j^{(p)} (-1)$$

El peso de sesgo se ajusta igual, con la entrada fija -1 .

La generalización: una capa p cualquiera

$$\Delta w_{ji}^{(p)}(n) = \eta \langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \rangle (1 + y_j^{(p)})(1 - y_j^{(p)}) y_i^{(p-1)}(n)$$

Una sola fórmula para **todas** las capas, tenga la red tres o veinte. Los tres índices: p es la capa donde estoy parado, $p - 1$ de dónde viene la entrada, $p + 1$ la capa siguiente. Léida en voz alta: **velocidad de aprendizaje, por error, por derivada de la activación en la capa actual, por entrada**. Y acá $\eta = \mu/2$, porque absorbe el $\frac{1}{2}$ de la derivada.

$$\langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \rangle = \sum_k \delta_k^{(p+1)} w_{kj}^{(p+1)}$$

El producto interno desplegado. k recorre las neuronas de la **capa siguiente** y j queda fijo, así que $\mathbf{w}_j^{(p+1)}$ es la **columna** j de esa matriz: los pesos que **salen** de la neurona j , no los que entran. Hacia adelante se suma sobre el segundo índice de w_{kj} ; hacia atrás, sobre el primero. Es la misma matriz transpuesta, y de ahí el nombre del método.

IDA: la neurona k usa su FILA

$$v_k = \sum_j w_{kj} y_j \quad (\text{lo que le ENTRA a } k)$$

	sesgo	j=1	j=2	j=3
k=1	w_{10}	w_{11}	w_{12}	w_{13}
k=2	w_{20}	w_{21}	w_{22}	w_{23}
k=3	w_{30}	w_{31}	w_{32}	w_{33}

VUELTA: la neurona j usa su COLUMNA

$$\sum_k \delta_k w_{kj} \quad (\text{lo que SALE de } j)$$

	sesgo	j=1	j=2	j=3
k=1	w_{10}	w_{11}	w_{12}	w_{13}
k=2	w_{20}	w_{21}	w_{22}	w_{23}
k=3	w_{30}	w_{31}	w_{32}	w_{33}

La misma matriz $\mathbf{W}^{(p+1)}$: filas = neurona que RECIBE (k), columnas = neurona que ENVÍA (j). Ida por filas, vuelta por columnas = transpuesta.

Figura 7: Ida: fila. Vuelta: columna. Cambiar una por otra es el error más común del tema.

Los dos bordes

La fórmula general es idéntica en todas las capas y sólo falla en los extremos, porque ahí **falta un vecino**.

$$\Delta w_{ji}^{(III)} = \eta \underbrace{(d_j - y_j^{(III)})}_{e_j} (1 + y_j^{(III)})(1 - y_j^{(III)}) y_i^{(II)}$$

Capa de salida: no existe $p + 1$, así que el producto interno se reemplaza por el **error verdadero**. Todo lo demás queda igual.

$$\Delta w_{ji}^{(I)} = \eta \langle \delta^{(II)}, \mathbf{w}_j^{(II)} \rangle (1 + y_j^{(I)})(1 - y_j^{(I)}) x_i$$

Primera capa: no existe $p - 1$, así que la «salida de la capa anterior» es la **entrada de la red**.

$$\delta_j^{(III)} \longrightarrow \delta_j^{(II)} \longrightarrow \delta_j^{(I)}$$

Y el orden no es negociable: los δ se calculan **de la salida hacia atrás**, porque cada capa necesita los de la siguiente. El de la capa de salida es el único que se puede calcular sin depender de nadie.

El corchete, desglosado un nivel

$$\langle \delta^{(III)}, \mathbf{w}_j^{(III)} \rangle = \frac{1}{2} \sum_k (d_k - y_k^{(III)})(1 + y_k^{(III)})(1 - y_k^{(III)}) w_{kj}^{(III)}$$

Reemplazando $\delta_k^{(III)}$ por su fórmula y sacando el $\frac{1}{2}$ afuera de la suma, el corchete queda escrito **sólo con cosas ya calculadas**: las salidas deseadas, las salidas de la red y los pesos.

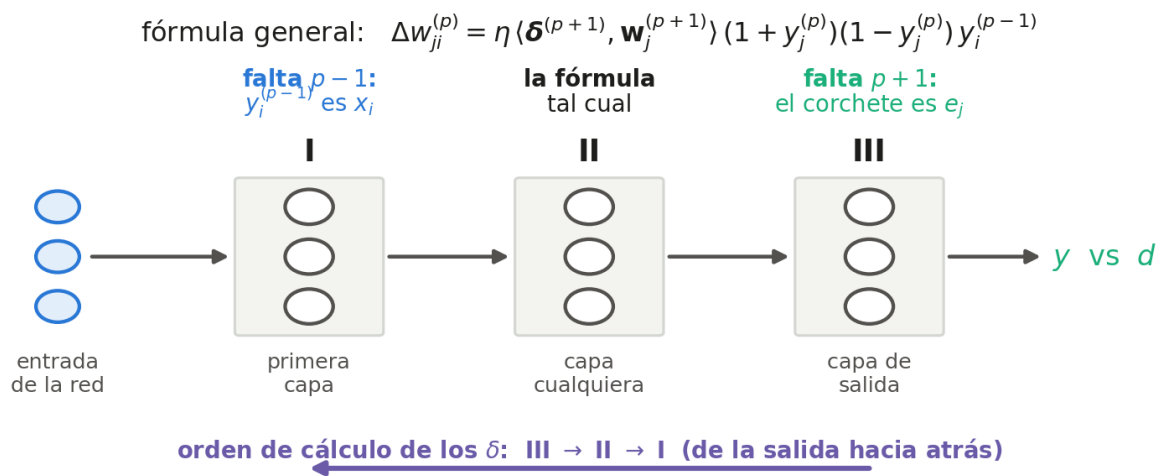


Figura 8: La fórmula general sólo falla donde falta un vecino.

OJO — dos derivadas de activación distintas, y el $\frac{1}{2}$

La derivada de adentro del corchete va evaluada en k (capa siguiente); la de afuera, en j (capa actual). Escribirlas con el mismo índice es el error más común del tema. Y de los dos $\frac{1}{2}$: el de adentro es el del $\delta^{(p+1)}$ y se escribe; el de la capa actual está absorbido en η .

Y hasta acá no más. Con más capas, cada $\delta^{(p+1)}$ tiene adentro otro corchete, y desplegar eso son cuatro renglones ilegibles. La gracia del δ es justamente **no** desplegarlo: se calcula una vez por neurona y en cada capa se lo trata como un número que ya está.

4. Redes de base radial

SI TE DICEN — «contame sobre las redes de base radial»

«Es una red de dos capas donde las neuronas ocultas son **gaussianas** en vez de sigmoideas: cada una tiene un centro, responde fuerte si la entrada está cerca y casi nada si está lejos. Son **detectores locales**, burbujas, en vez de semiplanos. La salida es una **combinación lineal** de esas respuestas. El entrenamiento es **mixto, en dos fases**. Primero, **sin mirar etiquetas**, se ubican los centros con k -medias: cada dato va al centro más cercano y cada centro se muda al promedio de sus datos, hasta que nadie cambie. Después, **con las etiquetas**, se entrenan los pesos de salida con LMS, usando las gaussianas como entradas. La ventaja: esa segunda fase es un **problema lineal**, más simple que retropropagar. Como variante, después se pueden ajustar también centros y anchos por gradiente.»

IDEA DE FONDO — en qué consiste

Cambiar de representación para que el problema se vuelva fácil. Tiene **dos fases con criterios distintos**: primero, **sin mirar las etiquetas**, se ubican unas cuantas «burbujas» gaussianas donde hay datos; después, **mirando las etiquetas**, se decide cuánto pesa cada burbuja en la salida. La imagen: cada neurona oculta es un detector local que se enciende cuando la entrada cae cerca de su centro y se apaga si está lejos — al revés de la sigmoide, que responde a todo un semiespacio. Y la clave del método: una vez fijadas las burbujas, **la salida es una combinación lineal**, así que la segunda mitad es un problema lineal, sin capas ocultas que derivar.

Dos fases con criterios distintos: la primera no supervisada ubica las funciones, la segunda supervisada las combina.

Arquitectura

$$\varphi_j(\mathbf{x}_\ell) = e^{-\|\mathbf{x}_\ell - \mu_j\|^2 / 2\sigma_j^2}$$

Cada neurona oculta responde según **qué tan cerca** está la entrada de su centro μ_j : máximo si coincide, y cae con la distancia a una velocidad que fija σ_j . Es una activación local, al revés que la sigmoide, que responde a todo un semiespacio.

Sigmoide: global
se enciende en **TODO** un semiplano



Radial: local
se enciende sólo **CERCA** de su centro

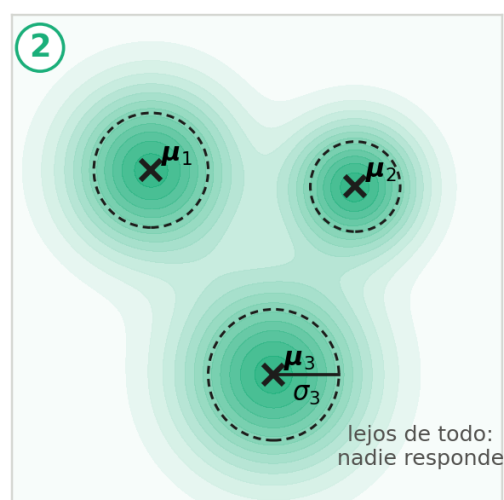


Figura 9: La forma de la activación es toda la diferencia entre los dos métodos: semiespacios contra burbujas.

$$y_k(\mathbf{x}_\ell) = \sum_{j=1}^M w_{kj} \varphi_j(\mathbf{x}_\ell)$$

La salida es una **combinación lineal** de esas respuestas. Y ahí está la clave del método: fijadas las funciones radiales, la salida es lineal en los pesos, así que la segunda fase es un problema fácil.

La salida es una suma de campanas pesadas
cambiar un w_{kj} sólo sube, baja o da vuelta SU campana (línea fina: la campana sin pesar)

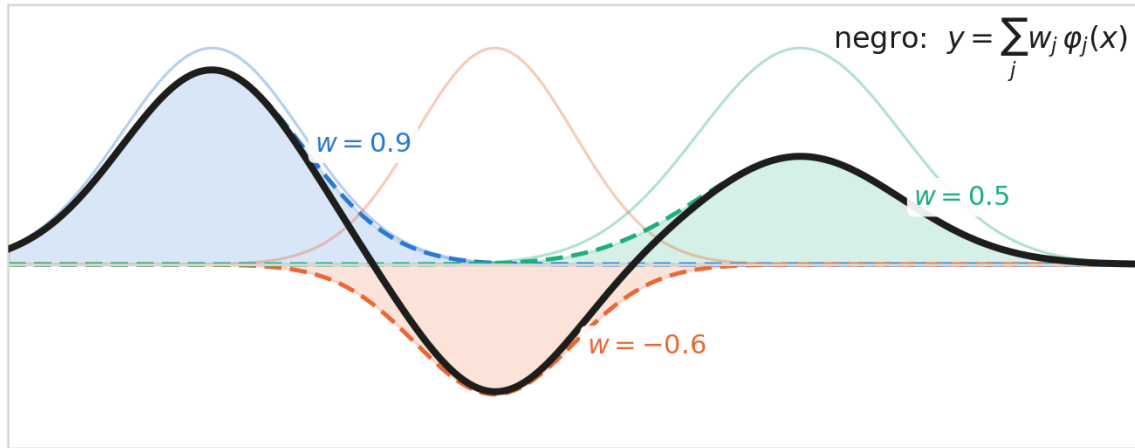


Figura 10: Cambiar un w_{kj} sube o baja una campana; la forma y la posición no se tocan.

Fase 1 — centros, no supervisada (k -medias)

$$J = \sum_j \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \mu_j\|^2$$

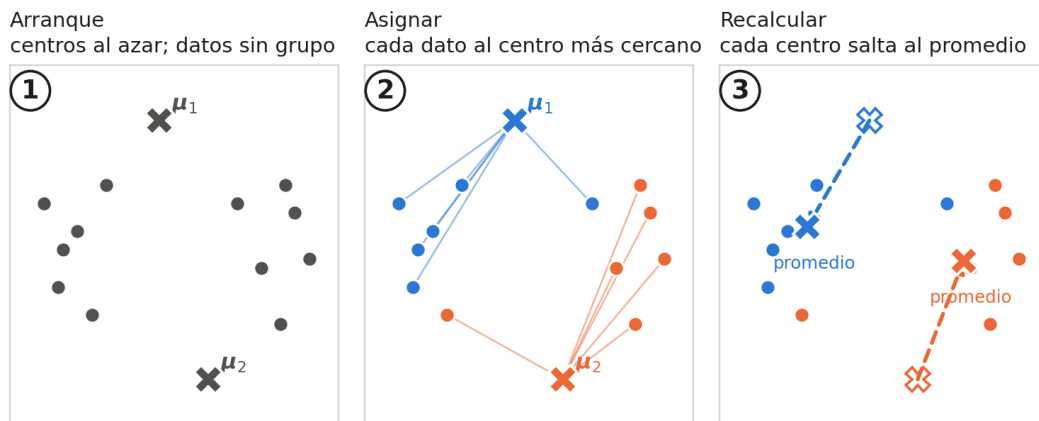
El criterio: que cada dato esté lo más cerca posible del centro de su grupo. Se minimiza alternando dos pasos.

$$\ell \in C_j \iff \|\mathbf{x}_\ell - \mu_j\|^2 < \|\mathbf{x}_\ell - \mu_i\|^2 \quad \forall i \neq j$$

Reasignación: cada dato se asigna al centro más cercano.

$$\mu_j = \frac{1}{|C_j|} \sum_{\ell \in C_j} \mathbf{x}_\ell$$

Recálculo: cada centro se muda al promedio de los datos que le tocaron. Se repite hasta que nadie cambia de grupo.



Los datos nunca se mueven (en 2 sólo cambian de color). Se repiten 2 y 3 hasta que ningún dato cambie de grupo.

Figura 11: Los patrones nunca se mueven: en el primer paso cambian de etiqueta, en el segundo salta el centro.

$$j^* = \arg \min_j \|\mathbf{x}_\ell - \mu_j(n)\| \quad \mu_{j^*}(n+1) = \mu_{j^*}(n) + \eta(\mathbf{x}_\ell - \mu_{j^*}(n))$$

La versión **en línea**, patrón por patrón: gana el centro más cercano y se corre un poco hacia el dato. Es la misma idea que el SOM pero sin vecindad.

Fase 2 — pesos, supervisada

$$e_k(n) = y_k(n) - d_k(n) \quad \xi(n) = \frac{1}{2} \sum_k e_k^2(n)$$

Recién acá aparecen las salidas deseadas. **Ojo con el signo:** acá el error se define al revés que en el perceptrón, $e = y - d$, y por eso la regla lleva $-\eta$. Es la misma cuenta que $+\eta(d - y)$.

$$\frac{\partial \xi}{\partial w_{kj}} = e_k(n) \varphi_j(n)$$

El gradiente es sencillo porque la salida es lineal en los pesos: no hay ninguna derivada de activación que arrastrar.

$$w_{kj}(n+1) = w_{kj}(n) - \eta e_k(n) \varphi_j(n)$$

Error por activación de la neurona oculta. Es LMS otra vez, con las φ_j haciendo de entradas.

Cómo se lee

$\varphi_j(\mathbf{x})$ se lee como **cuánto se parece la entrada al centro j** : vale 1 si cae justo encima y cae con la distancia a una velocidad que fija σ_j . Es un **detector local**, al revés de la sigmoide, que responde a todo un semiespacio. Y w_{kj} se lee como **cuánto pesa esa burbuja en la salida k** .

Por eso la frontera de una RBF son **burbujas** y la de un multicapa son **semiespacios cortados**: la diferencia entre los dos métodos está en la forma de la activación, no en el algoritmo.

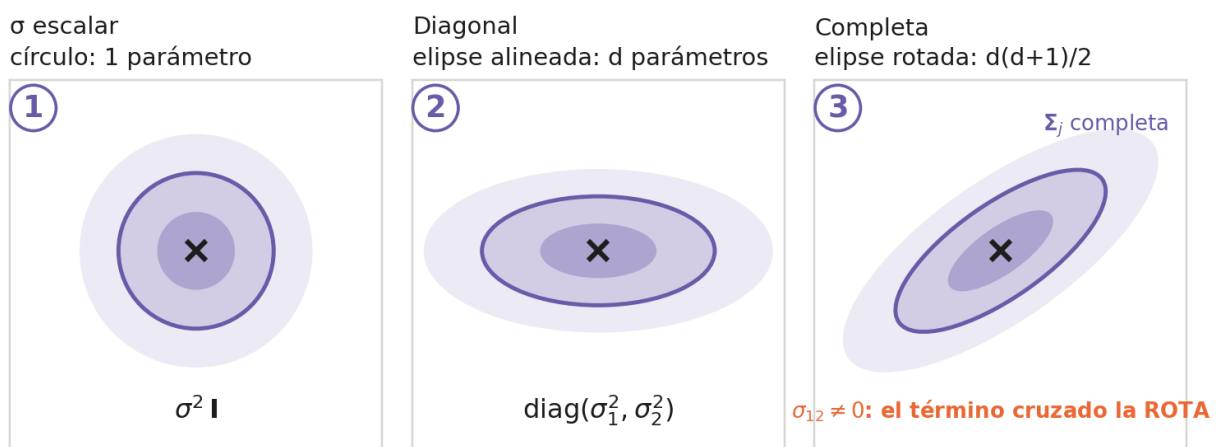


Figura 12: Si las burbujas tienen que ser elipses, el escalar σ_j se reemplaza por una matriz: los términos cruzados son los que la rotan.

PARA LA DEFENSA — la respuesta de treinta segundos

«Se cambia de representación para que el problema se vuelva fácil. Primero, sin mirar las etiquetas, se ubican unas gaussianas donde hay datos —eso es *k-medias*—. Después, con las etiquetas, se decide cuánto pesa cada una en la salida. Y la clave: fijadas las gaussianas, la salida es una combinación lineal de ellas, así que la segunda fase es un problema lineal, sin capas ocultas que derivar.» **Las repreguntas seguras:** por qué el entrenamiento es **mixto** (fase 1 no supervisada, fase 2 supervisada); qué pasa si las gaussianas tienen que ser elipses (se reemplaza el escalar σ_j por una matriz de covarianza); y el **método 2**, que arranca con el método 1 y después ajusta todo —centros y anchos incluidos— por gradiente.

5. Hopfield

SI TE DICEN — «contame sobre la red de Hopfield»

«Es una **memoria asociativa**: guarda patrones de ± 1 , y si le das uno incompleto o con ruido te devuelve el guardado más parecido. Es **una sola capa**, todas las neuronas conectadas con todas, con **pesos simétricos y sin autoconexiones**. Tiene dos fases. **Almacenar no itera**: los pesos salen de una cuenta con la **regla de Hebb**, el promedio de $x_j x_i$ sobre los patrones, que mide cuánto coinciden esas dos neuronas. **Recuperar sí itera**: cargo el patrón sucio como estado inicial, sorteo una neurona, le pongo el signo de su campo local, y repito hasta N pasos seguidos sin cambios. **Siempre converge**, porque con pesos simétricos hay una **energía que nunca sube**. Sus límites: capacidad baja, del orden de $N/(2 \ln N)$ patrones, y **estados espurios**, como el negativo de cada patrón.»

IDEA DE FONDO — en qué consiste

Una memoria que se consulta **por contenido**: le das un dato roto y te devuelve el dato entero. Tiene **dos fases**: **almacenar**, que es una sola cuenta y no itera, y **recuperar**, que itera hasta quedarse quieta. La idea de fondo: el recuerdo **no se guarda en ningún lugar**, se guarda como **acuerdos entre las partes del recuerdo** — para cada par de neuronas, si suelen coincidir o suelen oponerse. Por eso están **todas conectadas con todas**, en una sola capa. Y recuperar es **una votación ponderada que se repite**: se sortea una neurona, las demás le dicen qué valor debería tener, **el peso del cable es cuánto vale cada voto**, gana la mayoría, y se repite hasta que nadie cambia.

Datos de partida

$$\mathbf{x}_k^* \in \{-1, +1\}^N, \quad k = 1, \dots, P$$

P patrones para guardar, cada uno un vector de N componentes que valen -1 o $+1$. Esos N lugares son las N neuronas.

Fase 1 — Almacenamiento

$$w_{ji} = \frac{1}{N} \sum_{k=1}^P x_{kj}^* x_{ki}^* \quad \forall i \neq j$$

Para cada par de neuronas se recorren los patrones multiplicando el valor que tiene una por el valor que tiene la otra. Como son ± 1 , cada producto da $+1$ si coinciden y -1 si se oponen: el peso mide **en cuántos patrones coincidieron menos en cuántos difirieron**.

$$w_{jj} = 0 \quad \forall j$$

Ninguna neurona se conecta consigo misma. Se impone: la fórmula daría P/N y esa neurona se haría caso a sí misma por encima de todas las demás.

$$w_{ji} = w_{ij}$$

La conexión entre dos neuronas es una sola. Con la regla de arriba se cumple solo, porque el producto no distingue el orden.

Fase 2 — Recuperación

$$\mathbf{y}(0) = \mathbf{x}$$

El patrón incompleto o con ruido se escribe directamente como estado inicial de las neuronas.

$$j^* = \text{rnd}(N)$$

Se sortea cuál neurona se actualiza en este paso.

$$v_{j^*}(n) = \sum_{i=1}^N w_{j^*i} y_i(n-1)$$

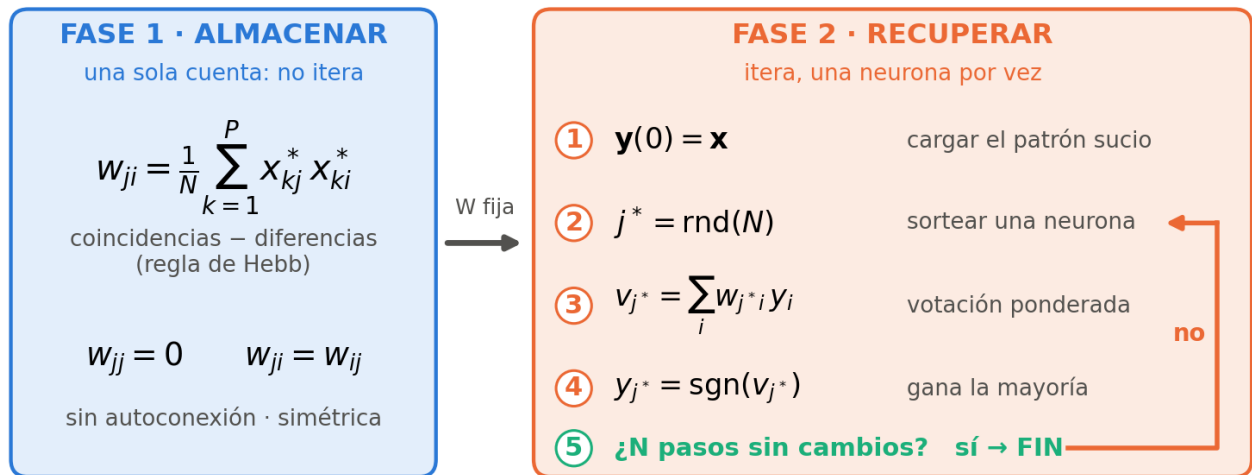


Figura 13: Escribir la memoria es una cuenta; leerla es dejar correr la red. Los dos carriles no se mezclan.

El campo local: el valor actual de cada una de las demás, multiplicado por el peso que la une con la elegida, todo sumado. Es una votación ponderada, donde el peso es cuánto vale cada voto.

$$y_{j^*}(n) = \text{sgn}(v_{j^*}(n)), \quad \text{sgn}(x) = \begin{cases} +1 & x > 0 \\ y_{j^*}(n-1) & x = 0 \\ -1 & x < 0 \end{cases}$$

El nuevo valor es el signo de ese total: gana la mayoría. En el empate la neurona conserva lo que tenía.

$$y_j(n) = y_j(n-1) \quad \forall j \neq j^*$$

Las demás quedan como estaban: se toca **una sola** por paso.

$$y(n) = \dots = y(n-N) \implies \text{fin}$$

Se corta tras N actualizaciones seguidas sin cambios. Recién ahí nada puede moverse después, porque a cada neurona le entraría lo mismo que ya le entró.

Convergencia

$$E(y) = -\frac{1}{2} \sum_j \sum_i w_{ji} y_i y_j$$

Cada par de neuronas aporta un término que baja la energía cuando el par está como su peso “quiere”. El $\frac{1}{2}$ compensa que la doble suma cuenta cada par dos veces.

$$\Delta E = -(y_{j^*}(n) - y_{j^*}(n-1)) v_{j^*}(n)$$

Al actualizar una sola neurona, los únicos términos que se mueven son los que la contienen. **Esto vale porque los pesos son simétricos:** si no, el par (i, j) aportaría dos términos distintos.

$$\Delta E = 2 y_{j^*}(n-1) v_{j^*}(n) < 0 \quad \text{si cambió} \quad \Delta E = 0 \quad \text{si no cambió}$$

Si la neurona se dio vuelta fue porque su valor viejo no coincidía con el signo del campo: tienen signos distintos,

el producto es negativo y la energía bajó.

$$\boxed{\Delta E \leq 0} \wedge 2^N < \infty \implies \text{converge en pasos finitos}$$

La energía nunca sube, está acotada por abajo y hay finitos estados: no puede bajar para siempre.

$$E(-\mathbf{y}) = E(\mathbf{y}) \quad P_{\max} = \frac{N}{2 \ln N}$$

Dar vuelta todas las neuronas no cambia la energía: por cada memoria guardada queda su negativo como estado espúreo.

La capacidad, en cambio, crece más lento que N , mientras la cantidad de pesos crece como N^2 : guardar muchos patrones sale caro.

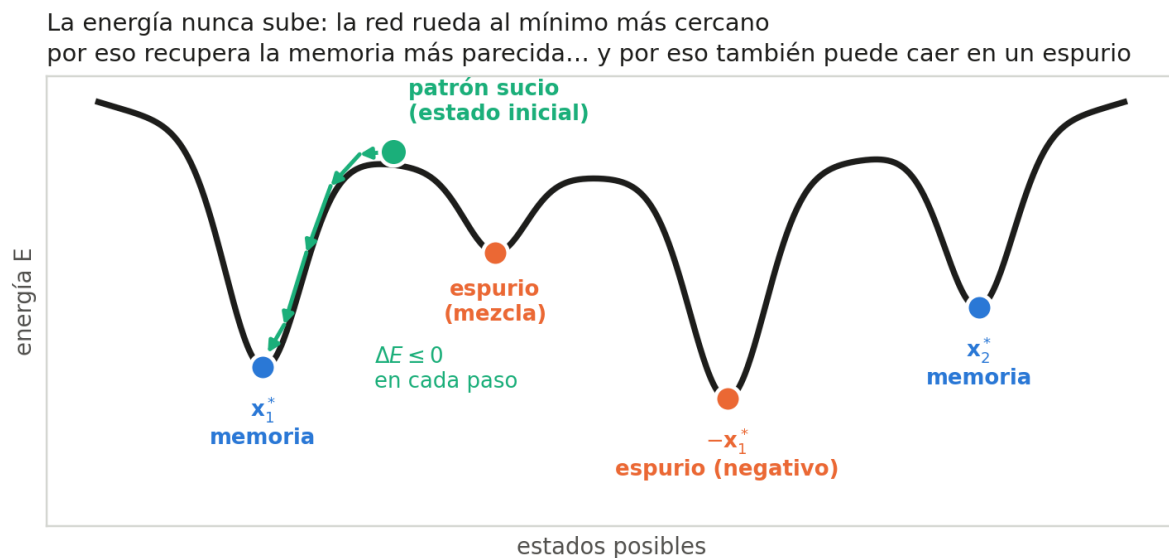


Figura 14: La red rueda cuesta abajo hasta el mínimo más cercano: por eso recupera la memoria más parecida, y por eso también puede caer en uno espurio.

Cómo se leen las tres piezas

w_{ji} — *coincidencias menos diferencias*: en cuántos patrones esas dos neuronas tuvieron el mismo valor, menos en cuántos difirieron. Es **el acuerdo** entre dos posiciones del dato.

v_j — *una votación ponderada*: cada neurona opina sobre cuánto debería valer j , y **el peso del cable es cuánto vale cada voto**. Positivo dice «tenemos que coincidir», negativo «tenemos que oponernos», cero «no opino».

E — *cuánta incomodidad hay en la red*: cuántos pares están en contra de lo que su peso quiere. Cada par contento la baja, cada par a disgusto la sube.

PARA LA DEFENSA — la respuesta de treinta segundos

«Es una memoria asociativa: se consulta por contenido y no por dirección. El recuerdo no se guarda en ningún lugar, se guarda como acuerdos entre las partes del recuerdo, y por eso están todas las neuronas conectadas con todas. Almacenar es una sola cuenta —Hebb—, y recuperar es una votación ponderada que se repite hasta que nadie cambia. Y siempre termina, porque hay una energía que nunca sube.» **Las repreguntas seguras**: por qué es **hebbiano** (el peso crece cuando dos neuronas se activan juntas, no hay salida deseada en ningún momento, y el ajuste depende sólo de la correlación entre las dos actividades); por qué la diagonal es cero; qué pasa si los pesos no son simétricos (puede oscilar, y se cae la demostración); y qué es un estado espúreo.

6. BPTT

SI TE DICEN — «contame sobre BPTT»

«Es cómo se entrena una **red recurrente**, donde la salida de un instante vuelve a entrar en el siguiente. La idea es **desenrollarla en el tiempo**: T instantes son T copias en fila, o sea una red profunda, y le aplico retropropagación. Primero recorro toda la secuencia **hacia adelante guardando** campos locales y salidas; después calculo los errores y **vuelvo hacia atrás en el tiempo** propagando los deltas con la **transpuesta** de la matriz de recurrencia. Dos diferencias con el multicapa: como **las copias comparten los mismos pesos**, los aportes de todos los instantes **se suman** sobre el mismo peso; y los pesos se actualizan **recién al final de la secuencia**. Como la memoria crece con T , se puede **truncar** la vuelta a pocos instantes, a costa de perder dependencias largas. Elman y Jordan son el caso truncado a un solo paso.»

IDEA DE FONDO — en qué consiste

Entrenar una red recurrente con back-propagation, **desenrollando el tiempo**: si la red se realimenta durante T instantes, se la dibuja como T copias en fila y se la trata como una red profunda. Tres partes: **ida** por toda la secuencia guardando todo, **medir los errores**, y **vuelta en el tiempo** acumulando gradientes. Las dos diferencias con el multicapa, que son de donde salen todas las preguntas: **el tiempo hace de profundidad**, y **las copias comparten los mismos pesos** — por eso los aportes de todos los instantes **se suman** sobre el mismo peso en vez de ajustar cada capa por separado.

Datos de partida

$$\{(\mathbf{x}_t, \mathbf{d}_t)\}_{t=0}^{T-1}$$

Una secuencia, con salida deseada en los instantes que la tengan. No todos necesitan tenerla.

Inicialización

$$\mathbf{W}^I, \mathbf{W} \sim \text{al azar, chicos} \quad \mathbf{y}_{-1} = \mathbf{0} \quad \Delta \mathbf{W} = \Delta \mathbf{W}^I = \mathbf{0}$$

Pesos chicos para que las neuronas no arranquen saturadas con $\varphi' \approx 0$. Estado previo nulo y acumuladores en cero.

Paso hacia adelante

$$\mathbf{v}_t = \mathbf{W}^I \mathbf{x}_t + \mathbf{W} \mathbf{y}_{t-1} \quad \mathbf{y}_t = \varphi(\mathbf{v}_t)$$

Lo que entra tiene dos partes: la entrada de ese instante y la salida de la propia red en el anterior. Ésa es toda la recurrencia, y la **misma** $\mathbf{W}$ se usa en todos los instantes.

$$\text{guardar } \mathbf{v}_t, \mathbf{y}_t \quad \forall t$$

La vuelta va a necesitar $\varphi'(\mathbf{v}_t)$ de cada instante. Es la diferencia práctica más grande con el multicapa, y la razón de que la memoria crezca con T .

Error

$$\mathbf{e}_t = \mathbf{y}_t - \mathbf{d}_t \quad E = \frac{1}{2} \sum_t \sum_k e_{t,k}^2$$

El error de la secuencia es la suma de los errores de todos los instantes. Los que no tienen salida deseada aportan cero. Igual que en la RBF, $e = y - d$: por eso la actualización del final **resta**.

Gradiente — forma derivada

$$\delta_{t,j} = e_{t,j} \varphi'(v_{t,j})$$

En el instante donde se midió el error, el delta es directo: igual que la capa de salida del multicapa.

$$\delta_{\tau,j} = \left(\sum_k w_{kj} \delta_{\tau+1,k} \right) \varphi'(v_{\tau,j}) \quad \tau < t$$

Hacia atrás en el tiempo, el delta se arrastra: los deltas del instante siguiente, pesados por las conexiones, por la derivada de la activación de esta neurona en este instante. **El tiempo hace de profundidad.** Ojo con el índice: hacia adelante $v_{\tau+1,k}$ recibe $w_{kj} y_{\tau,j}$, así que hacia atrás se suma sobre k con w_{kj} — es la columna j , lo mismo que $\mathbf{W}^T$ en la forma optimizada.

IDA: la MISMA W en todas las flechas (y se guarda todo)

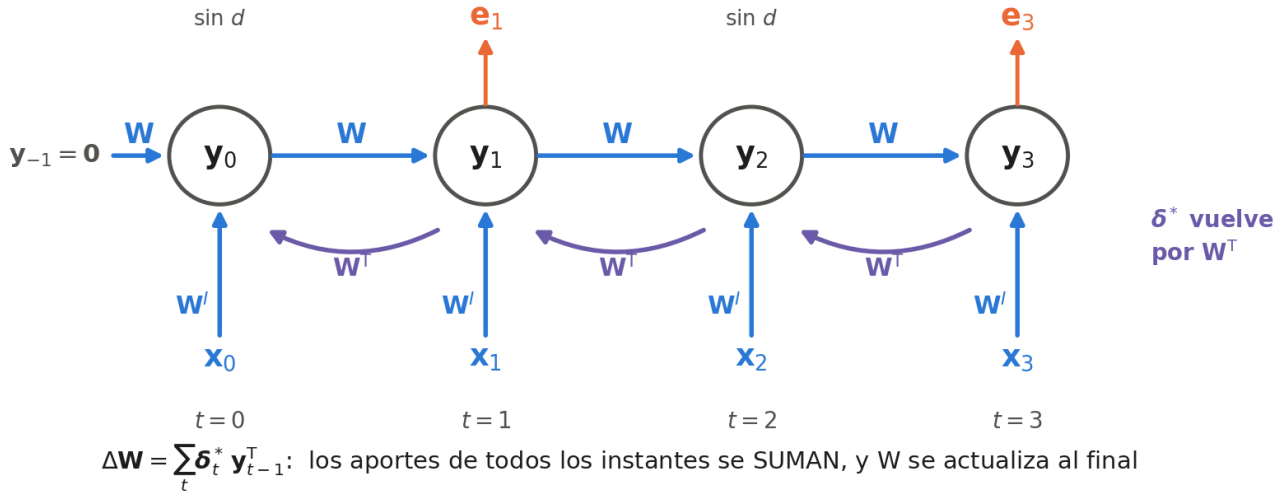


Figura 15: Desenrollada, la recurrencia es una red profunda con una copia por instante y la misma $\mathbf{W}$ repetida.

$$\frac{\partial E_t}{\partial w_{ji}} = \sum_{\tau=0}^t \delta_{\tau,j} y_{\tau-1,i} \quad \frac{\partial E_t}{\partial w_{ji}^I} = \sum_{\tau=0}^t \delta_{\tau,j} x_{\tau,i}$$

El error de un instante se reparte sobre **todos** los anteriores, porque el mismo peso participó en todos, y los aportes se **suman**. Para los pesos de entrada los deltas son los mismos: cambia sólo el último factor.

Gradiente — forma optimizada

$$\delta_T^* = \mathbf{0} \quad \delta_t^* = [\mathbf{e}_t + \mathbf{W}^T \delta_{t+1}^*] \odot \varphi'(\mathbf{v}_t)$$

El delta acumulado junta el error medido en ese instante con todo lo arrastrado de los posteriores, ya sumado. Se recorre de $T - 1$ a 0, al revés que la propagación. La **transpuesta** es lo que distingue la ida de la vuelta.

$$\Delta \mathbf{W} += \delta_t^* \mathbf{y}_{t-1}^T \quad \Delta \mathbf{W}^I += \delta_t^* \mathbf{x}_t^T$$

En el mismo barrido se acumulan los dos gradientes. Da **exactamente el mismo** gradiente que la forma derivada, reorganizado, pero en **un solo barrido**: el costo baja de $O(T^2)$ a $O(T)$.

Actualización

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \Delta \mathbf{W} \quad \mathbf{W}^I \leftarrow \mathbf{W}^I - \eta \Delta \mathbf{W}^I$$

Los pesos se tocan **al final de la secuencia**. Si se actualizaran en el medio del barrido, los deltas que faltan usarían una $\mathbf{W}$ distinta de la que se usó hacia adelante.

Truncado

$$\tau \geq \max(0, t - P) \quad O(T^2) \rightarrow O(T P)$$

Truncar limita la vuelta a P instantes: el costo pasa a $O(T P)$, pero se pierden las dependencias más largas que P . Las dos cosas abaratan el cálculo, con una diferencia que se pregunta: **truncar cambia el gradiente (es una aproximación); optimizar con δ^* no (da el mismo, exacto).**

Arquitecturas con retardo

$$\tilde{\mathbf{x}}_t = (\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-D})$$

TDNN: la entrada se amplía con sus retardos y la red vuelve a ser hacia adelante. Toda la memoria está en la entrada, no en la red.

$$\mathbf{y}_t = \varphi(\mathbf{W}^I \mathbf{x}_t + \mathbf{W}^C \mathbf{c}_t), \quad \mathbf{c}_t = \mathbf{y}_{t-1}^{\text{oculta}} \text{ (Elman)} \quad \mathbf{c}_t = \mathbf{y}_{t-1}^{\text{salida}} \text{ (Jordan)}$$

La capa de contexto guarda la salida anterior —de la oculta en Elman, de la salida en Jordan— y se trata como una entrada más con valores congelados. Con eso se entrena con back-propagation común: es BPTT truncada a un solo paso.

Cómo se lee

$\delta_{\tau,j}$ es el mismo delta del multicapa, con «capa siguiente» reemplazado por «**instante** siguiente»: el tiempo hace de profundidad. δ_t^* es el delta **acumulado**: ya trae sumados los errores de todos los instantes posteriores, y por eso alcanza un solo barrido.

Y la suma sobre τ se lee así: **el error de un instante se reparte sobre todos los instantes anteriores, porque el mismo peso participó en todos**. Los aportes **se suman** — no se promedian.

PARA LA DEFENSA — la respuesta de treinta segundos

«Es back-propagation con el tiempo haciendo de profundidad. Si la red se realimenta durante T instantes, la dibujo como T copias en fila y la trato como una red profunda. Con dos diferencias: las copias comparten los mismos pesos, así que los aportes de todos los instantes se suman sobre el mismo peso; y hay que guardar toda la secuencia hacia adelante, porque la vuelta necesita la derivada de la activación de cada instante.» **Las repreguntas seguras:** los aportes se **suman**, no se promedian; hacia adelante va $\mathbf{W}$ y hacia atrás $\mathbf{W}^T$; los pesos se actualizan **al final** de la secuencia, nunca en el medio del barrido; y **truncar no es lo mismo que optimizar** — truncar cambia el gradiente, el δ^* da exactamente el mismo.

7. SOM

SI TE DICEN — «contame sobre el SOM»

«El mapa de Kohonen es una red **no supervisada** que acomoda prototipos sobre los datos y **además los deja ordenados**. Cada neurona tiene un vector de pesos **en el mismo espacio que las entradas** y un lugar en una **grilla**, normalmente 2D. Por cada patrón, dos pasos: **competencia** —gana la neurona más cercana al dato— y **adaptación** —la ganadora y sus **vecinas de la grilla** se acercan al dato una fracción η —. Esa vecindad es lo que lo separa de k -medias: produce el **ordenamiento topológico**, neuronas vecinas en el mapa representan zonas vecinas de los datos. Se entrena en **tres etapas**: ordenamiento (vecindad grande, η alta), transición, y ajuste fino (vecindad cero, η chica). Se corta por épocas, porque no hay error que medir. Sirve para agrupar y visualizar datos de alta dimensión y, etiquetando las neuronas al final, para clasificar.»

IDEA DE FONDO — en qué consiste

Acomodar unos pocos prototipos para que representen a muchos datos, y **además queden ordenados entre sí**. Dos partes por cada patrón: **competir** (gana el prototipo más cercano) y **adaptar** (el ganador se corre un poco hacia el dato). La imagen clave: los pesos de cada neurona **viven en el mismo espacio que los datos**, así que son un punto que se va mudando hacia donde hay datos. Y lo que lo hace un *mapa* y no una simple cuantización: **cuando el ganador se mueve, arrastra a sus vecinos de la grilla**. Por eso neuronas contiguas terminan representando zonas contiguas del espacio de entrada.

Datos de partida

$$\{\mathbf{x}(n)\} \subset \mathbb{R}^M, \quad \text{sin etiquetas}$$

Sólo vectores de entrada. El entrenamiento es **no supervisado**: la red se organiza a partir de cómo están distribuidos los datos.

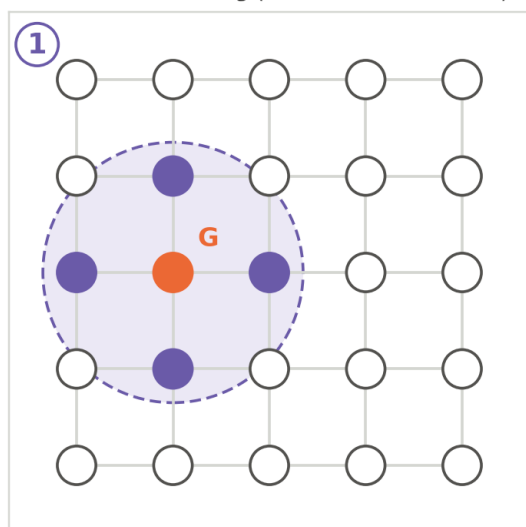
Arquitectura

$$\mathbf{w}_j \in \mathbb{R}^M, \quad j = 1, \dots, N$$

Los pesos de cada neurona viven **en el mismo espacio que las entradas**: son un punto del espacio de datos, no coeficientes. Por eso tiene sentido la distancia entre una entrada y una neurona.

Las neuronas están además en una grilla, que no tiene nada que ver con $\mathbb{R}^M$: define **quién es vecino de quién**, y es lo único que la topología aporta.

GRILLA: acá se mide la VECINDAD
violeta: el entorno Λ_G (no se deforma nunca)



DATOS: acá se mide la COMPETENCIA
gana la más cercana a $\mathbf{x}$; ella y sus vecinas se acercan

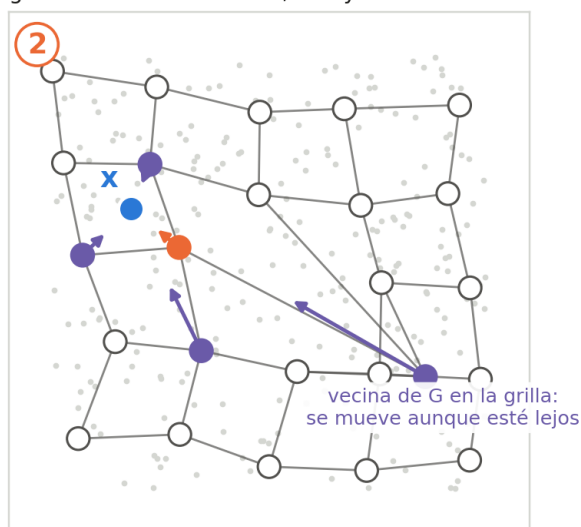


Figura 16: Los dos espacios del método. La vecindad se mide arriba; la competencia, abajo. Confundirlos es el error clásico del tema.

Inicialización

$$\mathbf{w}_j(0) \sim \text{al azar, chicos y distintos entre sí}$$

Distintos obligatoriamente: dos neuronas que arrancan en el mismo punto ganan y se mueven siempre juntas, y nunca se separan.

Competencia

$$j^*(n) = G(\mathbf{x}(n)) = \arg \min_{\forall j} \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

Se presenta una entrada y gana la neurona más cercana en el espacio de datos, midiendo con distancia euclídea.

Vecindad

$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

La influencia sobre cada neurona decae con su distancia **en la grilla** a la ganadora: la de al lado se mueve casi tanto como ella, una lejana casi nada. Con entorno uniforme, en cambio, es $\beta(n)$ adentro del radio y cero afuera.

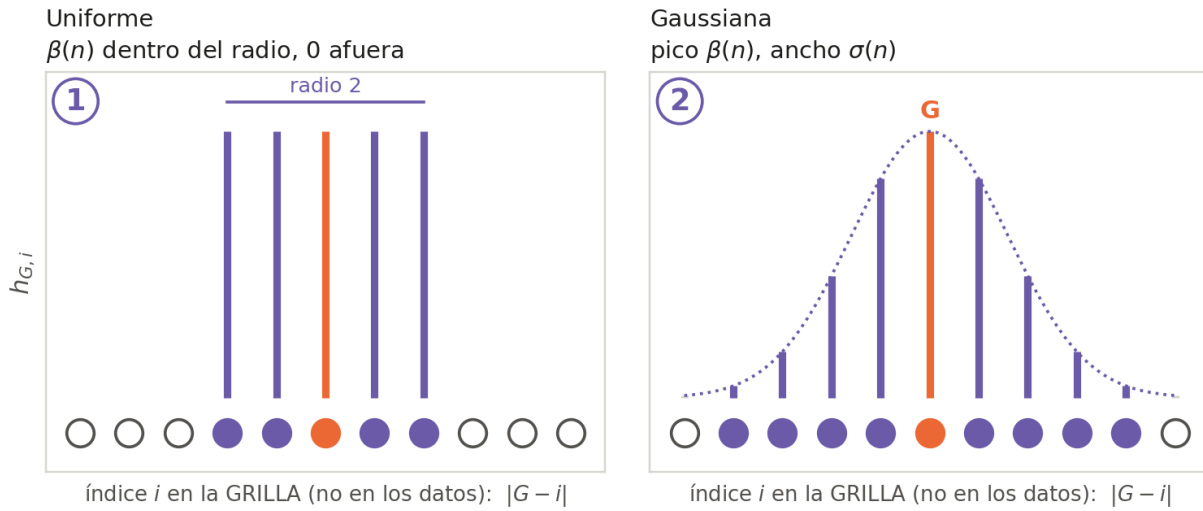


Figura 17: La influencia se mide sobre los índices de la grilla, no sobre los datos.

Adaptación

$$\mathbf{w}_j(n+1) = \begin{cases} \mathbf{w}_j(n) + \eta(n)(\mathbf{x}(n) - \mathbf{w}_j(n)) & y_j \in \Lambda_G(n) \\ \mathbf{w}_j(n) & y_j \notin \Lambda_G(n) \end{cases}$$

La ganadora y sus vecinas se mueven hacia la entrada: el paréntesis es el vector que va de la neurona al dato, y η dice qué fracción de ese camino se recorre. Las de afuera del entorno no se tocan. Ésta es la versión con **entorno uniforme**, la que se implementa.

$$\mathbf{w}_j(n+1) = \mathbf{w}_j(n) + h_{G,j}(n) (\mathbf{x}(n) - \mathbf{w}_j(n)) \quad \forall j$$

Con **vecindad gaussiana**, cada neurona recibe $h_{G,j}(n)$ en lugar de $\eta(n)$: la ganadora se mueve con el pico $\beta(n)$ y las demás cada vez menos según su distancia en la grilla. No hay corte en seco, y achicar $\sigma(n)$ es la forma continua de reducir el entorno.

Que se muevan **las vecinas de la grilla** es lo único que separa un SOM de una cuantización vectorial cualquiera: es lo que hace que neuronas contiguas terminen representando zonas contiguas del espacio de entrada.

Decaimiento — las tres etapas

$$\eta(n) \downarrow, \quad \Lambda_G(n) \downarrow$$

Etapa	Λ_G	η	Épocas
Ordenamiento	$\approx$ medio mapa	0,9 – 0,7	500 – 1000
Transición	$\rightarrow 1$, lineal	$\rightarrow 0,1$	≈ 1000
Ajuste fino	0	0,1 – 0,01, cte.	≈ 3000

Primero se ordena el mapa a lo grande con entorno amplio y pasos largos; después se afina con entorno nulo y pasos cortos. No hay error que mirar: se corta por cantidad de épocas.

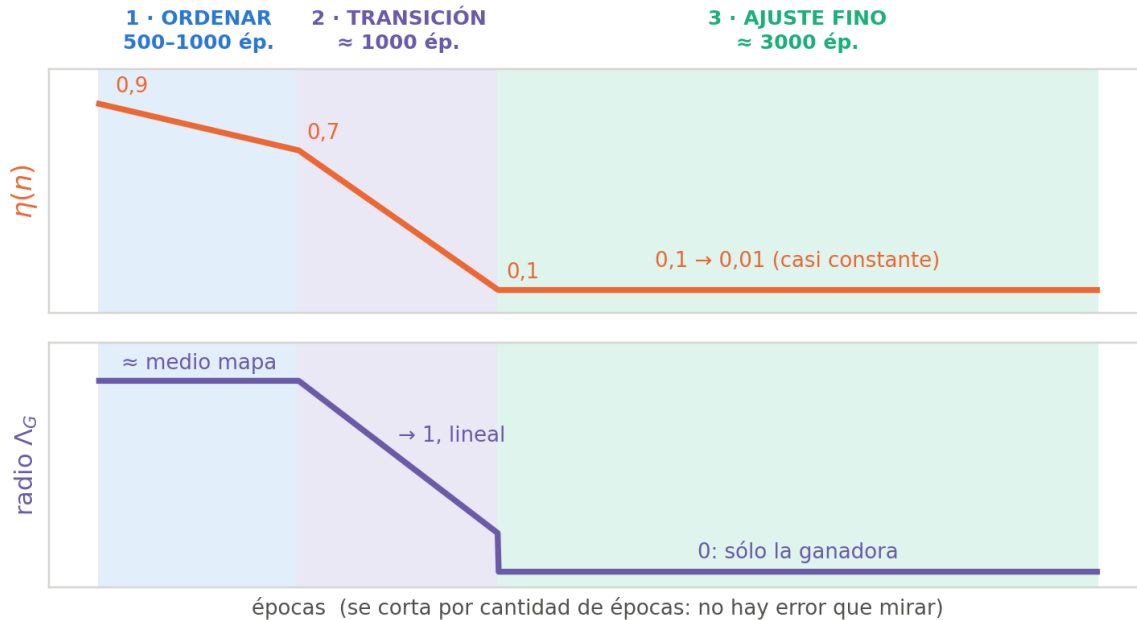


Figura 18: Las dos cantidades se apagan juntas: primero ordenar, después afinar.

Cómo se lee la adaptación

$\mathbf{x} - \mathbf{w}_j$ **no es un error contra una salida deseada**: es el vector que va de la neurona al dato, dos puntos del mismo espacio. Y η es **qué fracción de ese trecho se recorre**. Por eso el entrenamiento es no supervisado aunque la fórmula se parezca a la del gradiente.

Y hay **dos distancias distintas** en el algoritmo, que es lo que más se confunde: la de la competencia se mide **en el espacio de los datos**; la de la vecindad, **en la grilla**. Una neurona puede tener su peso lejísimos del patrón y actualizarse igual, sólo porque en el mapa está al lado de la ganadora. **Eso es exactamente lo que produce el ordenamiento topológico.**

PARA LA DEFENSA — la respuesta de treinta segundos

«Acomoda unos pocos prototipos para que representen a muchos datos, y además queden ordenados entre sí. Por cada patrón: gana el prototipo más cercano y se corre un poco hacia el dato. Lo que lo hace un mapa y no una simple cuantización es que el ganador arrastra a sus vecinos de la grilla, y por eso neuronas contiguas terminan representando zonas contiguas del espacio de entrada.» **Las repreguntas seguras:** por qué k -medias **no** tiene ordenamiento topológico (no hay vecindad: se mueve sólo el ganador); para qué sirve la etapa de ordenamiento y qué parámetros son clave (Λ_G grande y η alta); y por qué los pesos iniciales tienen que ser **distintos entre sí**.

8. LVQ1

SI TE DICEN — «contame sobre LVQ»

«Es la **versión supervisada** de la idea del SOM. Hay prototipos en el espacio de datos, **cada uno con una clase asignada**. Por cada patrón gana el más cercano y comparo su clase con la del dato: **si coincide, el prototipo se acerca; si no, se aleja**. Se mueve **sólo el ganador**, sin vecindad, porque no busco un mapa ordenado sino **ubicar bien la frontera entre clases**. Un dato nuevo toma la clase del prototipo más cercano. La variante **LVQ1-O** le da a cada prototipo **su propia tasa de aprendizaje**, que baja cuando acierta y sube cuando se equivoca, para que todos los patrones pesen parecido y el resultado no dependa del orden en que se presentaron.»

IDEA DE FONDO — en qué consiste

Lo mismo que el SOM pero **con etiquetas y con castigo**. Tres partes: **competir, ver si el ganador era de la clase correcta, y premiar o castigar**. Si acertó, el prototipo se acerca al dato; si se equivocó, **se aleja**. Eso es todo lo que cambia, y cambia el objetivo: al SOM le importa cubrir bien el espacio, a LVQ le importa **dónde queda la frontera entre clases**. Por eso acá no hay vecindad: se mueve sólo el ganador.

Datos de partida

$$\{(\mathbf{x}(n), d(n))\}, \quad \mathcal{C}(i) \text{ asignada a cada prototipo}$$

Acá **sí** hay etiquetas: LVQ es supervisado. Y cada prototipo tiene su clase asignada de antemano.

Ganador

$$c(n) = \arg \min_i \{\|\mathbf{x}(n) - \mathbf{m}_i(n)\|\}$$

Igual que en el SOM: gana el prototipo más cercano.

Signo

$$s(c, d, n) = \begin{cases} +1 & \mathcal{C}(c(n)) = d(n) \\ -1 & \mathcal{C}(c(n)) \neq d(n) \end{cases}$$

Acá está toda la diferencia con el SOM: se compara la clase del prototipo que ganó con la clase verdadera del dato.

Adaptación

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(c, d, n) \alpha [\mathbf{x}(n) - \mathbf{m}_c(n)]$$

El ganador se mueve **hacia** el dato si acertó la clase, y se **aleja** si se equivocó. El castigo es lo que no existe en el SOM, y es lo que empuja la frontera entre clases a su lugar.

$$\mathbf{m}_i(n+1) = \mathbf{m}_i(n) \quad \forall i \neq c$$

Se mueve **sólo el ganador**: no hay vecindad. A LVQ no le interesa la topología, le interesa la frontera de decisión.

Forma combinación

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)] \mathbf{m}_c(n) + s(n)\alpha(n) \mathbf{x}(n)$$

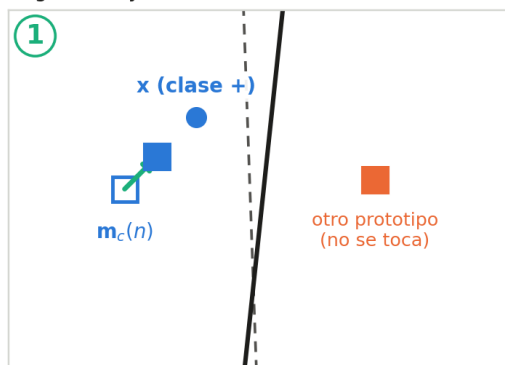
La misma regla reescrita: el prototipo nuevo es una combinación del viejo y del patrón que acaba de entrar. Escrita así se ve el problema del α constante.

LVQ1-O — el α óptimo

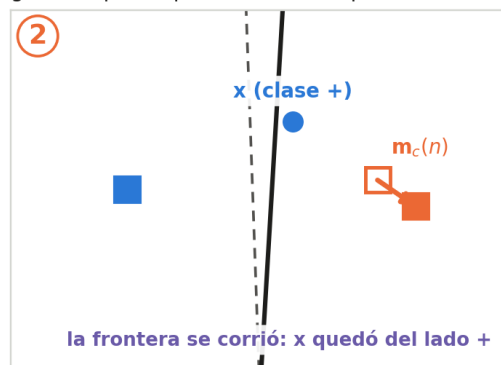
$$\alpha_c(n) = [1 - s(n)\alpha_c(n)] \alpha_c(n-1)$$

La condición que se impone: que el patrón anterior siga pesando lo mismo que el actual después de este paso. Con α constante no pasa — cada patrón viejo arrastra un factor más, y el prototipo termina dependiendo casi

Acierta la clase ($s = +1$): se ACERCA el ganador y x son de la misma clase



Se equivoca ($s = -1$): se ALEJA ganó un prototipo de la clase - para un x de clase +



$$\mathbf{m}_c \leftarrow \mathbf{m}_c + s \alpha (\mathbf{x} - \mathbf{m}_c) \quad \cdot \quad \text{línea punteada: frontera antes; línea llena: frontera después} \quad \cdot \quad \text{sólo se mueve el ganador}$$

Figura 19: Todo LVQ está en el signo.

sólo del final del archivo, que es un orden arbitrario.

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n) \alpha_c(n-1)}$$

Despejando queda esto: un α propio de cada prototipo, que decrece solo cuando acierta y crece cuando se equivoca. **Hay que saturarlo en 1** (la advertencia de la cátedra: «no sobrepasar $\alpha > 1$ »). Con $s = -1$ el denominador es $1 - \alpha_c(n-1) < 1$ y α crece; si se acumulan errores seguidos pasa de 1, el prototipo se pasa de largo del patrón en cada corrección y el algoritmo se vuelve inestable.

Cómo se lee

Todo LVQ está en el **signo**. Con $s = +1$ el prototipo se acerca al dato; con $s = -1$ **se aleja**. Ese castigo no existe en el SOM, y es lo que cambia el objetivo: al SOM le importa **cubrir bien el espacio**, a LVQ le importa **dónde queda la frontera entre clases**. Por eso acá no hay vecindad — la topología no le interesa.

PARA LA DEFENSA — la respuesta de treinta segundos

«Es el SOM con etiquetas y con castigo. Gana el prototipo más cercano, se mira si su clase era la correcta, y si acertó se acerca al dato y si se equivocó se aleja. Se mueve sólo el ganador: no hay vecindad, porque no busca un mapa ordenado sino una buena frontera de decisión.» **Las repreguntas seguras:** dónde entra la supervisión (**desde la primera iteración**, a diferencia del SOM etiquetado, donde entra recién después de entrenar); por qué no tiene ordenamiento topológico; y el problema del α constante — los primeros patrones pesan cada vez menos y el prototipo termina dependiendo del final del archivo, que es un orden arbitrario.

9. Capacidad de generalización y protocolo de evaluación

SI TE DICEN — «contame sobre generalización y evaluación»

«Generalizar es **funcionar bien con datos que la red no vio**, que es lo que realmente importa: el error de entrenamiento se puede llevar a cero memorizando. Como ese error no se puede calcular, **se estima con datos apartados**. Los datos se parten en **tres conjuntos disjuntos y estratificados**: entrenamiento ajusta los pesos, monitoreo decide hiperparámetros y cuándo parar, y prueba se mira **una sola vez** al final. El **sobreajuste** se ve en las curvas: entrenamiento sigue bajando mientras monitoreo toca un mínimo y sube; ahí se corta, **parada temprana**. Con pocos datos, **validación cruzada de k particiones**: se entrena k veces rotando qué bloque es de prueba y se reporta **media y varianza** del error. Para medir, la **matriz de confusión**: sensibilidad, especificidad, precisión, exactitud, F_1 . **Nunca una sola medida**, porque con clases desbalanceadas la exactitud engaña.»

IDEA DE FONDO — en qué consiste

Contestar «¿qué tan bien anda esto?» **sin engañarse**. Lo que interesa no es el error sobre los datos con los que entrené —ése lo puedo llevar a cero memorizando— sino el error sobre datos que la red **nunca vio**. Como ese error no se puede calcular, se **estima**, y todo el protocolo existe para que esa estimación no esté inflada. Tres partes: **partir los datos** en conjuntos con trabajos distintos, **monitorear** el entrenamiento para saber cuándo parar, y **medir** con el número adecuado. La trampa que hay que saber nombrar: si elegís la cantidad de épocas o la arquitectura mirando el conjunto de prueba, ese número **ya no mide generalización**, porque lo usaste para decidir.

No es un algoritmo de aprendizaje, pero es procedimiento y se pregunta igual.

Qué es generalizar

$$\varepsilon_{\text{gen}} = E[\varepsilon(\mathbf{x})]_{\mathbf{x} \sim \text{población}} \approx \varepsilon_{\text{prueba}}$$

La **capacidad de generalización** es el error esperado sobre toda la población de datos posibles, no sólo los que tengo. Como la población no se conoce, se la representa con un puñado de patrones **apartados desde el principio** y nunca usados para ajustar nada: ésa es la única estimación honesta que existe.

$$\varepsilon_{\text{entrenamiento}} \leq \varepsilon_{\text{prueba}}$$

En la práctica, casi siempre: el entrenamiento optimizó justo sobre esos patrones. (No es un teorema — con pocos patrones de prueba puede salir al revés por azar — pero si pasa seguido hay que sospechar de la partición.) Por eso **el error de entrenamiento nunca se reporta como resultado** — no mide nada que le importe a nadie. La distancia entre los dos números es lo que se llama **brecha de generalización**.

Memorizar contra generalizar

$$\varepsilon_{\text{entren}}(n) \downarrow \quad \text{mientras} \quad \varepsilon_{\text{monit}}(n) \uparrow$$

El síntoma del **sobreajuste**, y es lo único que hay que saber dibujar: dos curvas contra las épocas. La de entrenamiento baja siempre; la de monitoreo baja, toca un mínimo y **vuelve a subir**. A partir de ese punto la red dejó de aprender la regla y empezó a aprender el ruido de los patrones concretos que le tocaron.

Capacidad del modelo	Error de entrenamiento	Error de monitoreo	Nombre
Insuficiente	alto	alto	subajuste — ni siquiera puede representar la regla
Adecuada	bajo	bajo	el punto que se busca
Excesiva	≈ 0	alto	sobreajuste — memorizó

La capacidad se regula con la **cantidad de parámetros** (neuronas ocultas, capas, cantidad de centros en una RBF) y con **cuánto se entrena**. Con muchos parámetros y muchas épocas, cualquier red pasa por todos los puntos de entrenamiento — y no sirve para nada.

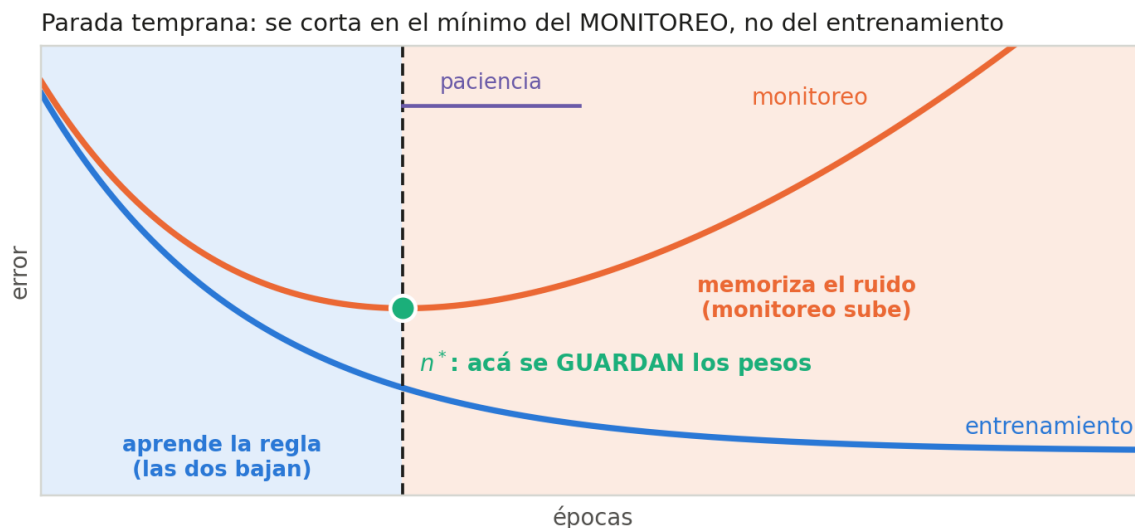


Figura 20: El único dibujo que hay que saber hacer del tema: dos curvas, un mínimo, y ahí se corta.

OJO — sobreajuste no es un error del algoritmo

El algoritmo hace exactamente lo que se le pidió: minimizar el error sobre el conjunto de entrenamiento. El problema es que **ése no era el objetivo real**. Por eso la solución no está adentro del algoritmo sino afuera: en cómo se parten los datos y cuándo se corta.

Las tres particiones

$$\mathcal{D} = \mathcal{D}_{\text{entren}} \cup \mathcal{D}_{\text{monit}} \cup \mathcal{D}_{\text{prueba}}, \quad \mathcal{D}_i \cap \mathcal{D}_j = \emptyset$$

El conjunto se parte en tres pedazos **disjuntos**. Disjuntos de verdad: un patrón repetido en dos conjuntos arruina la medición igual que si no se hubiera partido nada.

$$\mathcal{D}_{\text{entren}} \rightarrow \mathbf{W} \quad \mathcal{D}_{\text{monit}} \rightarrow \text{hiperparámetros y parada} \quad \mathcal{D}_{\text{prueba}} \rightarrow \text{el número final}$$

Cada uno tiene **un solo trabajo**, y ésta es la frase para el oral:

- **Entrenamiento** ($\approx 50\text{--}70\%$): es el único que entra en la regla de actualización. Sus patrones ajustan los pesos.
- **Monitoreo** o validación ($\approx 15\text{--}25\%$): **no ajusta pesos**, sólo se mide sobre él. Con eso se decide cuántas épocas, cuántas neuronas ocultas, qué μ , qué arquitectura.
- **Prueba** ($\approx 15\text{--}25\%$): se guarda en un cajón y se mira **una sola vez**, al final, con todo ya decidido. Es el que estima la generalización.

$$\text{proporción de clases en cada partición} = \text{proporción en } \mathcal{D}$$

Estratificación: las tres particiones tienen que respetar la proporción original de clases. Si el 10 % de los datos es de la clase positiva, el 10 % de cada partición también. Sin eso, una partición puede quedarse sin ejemplos de una clase y el número no significa nada.

Y antes de partir, **se mezcla**: un archivo ordenado por clase partido tal cual deja el conjunto de prueba con una sola clase adentro.

OJO — la normalización también se contamina

Media, desvío, mínimos y máximos para normalizar se calculan **sólo con el conjunto de entrenamiento** y después se aplican a los otros dos. Calcularlos sobre todo el conjunto es filtrar información de la prueba hacia el entrenamiento, y el error de prueba sale optimista.

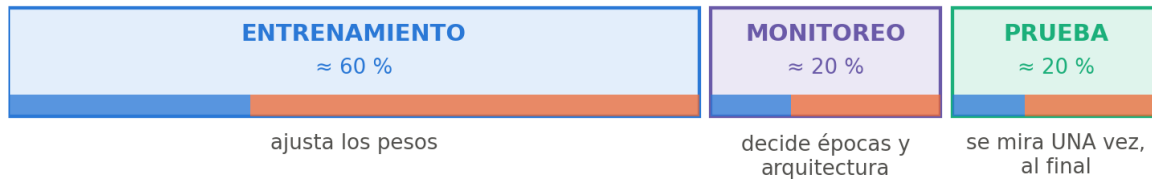
Monitoreo y parada temprana

$$n^* = \arg \min_n \varepsilon_{\text{monit}}(n) \quad \mathbf{W}^* = \mathbf{W}(n^*)$$

1 · mezclar el conjunto completo (35 % clase A, 65 % clase B)



2 · partir en tres, disjuntos y ESTRATIFICADOS (franja de abajo: la misma proporción en los tres)



3 · normalizar con media y desvío del ENTRENAMIENTO, y aplicarlos a los otros dos

Figura 21: Tres conjuntos disjuntos con un trabajo cada uno, y la misma proporción de clases en los tres.

Parada temprana: se entrena midiendo el error de monitoreo época por época, y se guarda la red del **mínimo de esa curva**, no la del final. El entrenamiento puede seguir un rato más para confirmar que el mínimo era el mínimo —eso es la *paciencia*— pero los pesos que quedan son los de n^* .

$$\varepsilon_{\text{monit}} \text{ decide} \implies \varepsilon_{\text{monit}} \text{ ya no mide}$$

Y acá está la razón de ser del tercer conjunto, que es lo que más se pregunta. El monitoreo **no entrena** —sus patrones nunca entran en Δw — pero **sí decide**: la época elegida es la que a él le convino. Después de usarlo para elegir, su error quedó tan sesgado como el de entrenamiento, sólo que menos. Por eso hace falta un conjunto de **prueba** que no haya participado de ninguna decisión.

Validación cruzada de k particiones (k -fold)

$$\mathcal{D} = \bigcup_{j=1}^k \mathcal{F}_j, \quad \mathcal{F}_i \cap \mathcal{F}_j = \emptyset, \quad |\mathcal{F}_j| \approx N/k$$

El conjunto se parte en k bloques (*folds*) disjuntos y del mismo tamaño, **estratificados** igual que antes. Se usa cuando hay **pocos datos**: apartar un 20 % fijo para prueba sería desperdiciar patrones y, además, el resultado dependería demasiado de cuáles tocaron.

$$\varepsilon_j = \varepsilon(\mathcal{F}_j) \quad \text{entrenando con } \mathcal{D} \setminus \mathcal{F}_j, \quad j = 1, \dots, k$$

Se entrena k veces. En la vuelta j , el bloque $\mathcal{F}_j$ hace de prueba y los otros $k - 1$ de entrenamiento. Así **cada patrón se usa para medir exactamente una vez** y para entrenar $k - 1$ veces. Cada vuelta **reinicializa los pesos de cero**: si no, la segunda red arrancaría sabiendo cosas del bloque que le toca medir.

$$\bar{\varepsilon} = \frac{1}{k} \sum_{j=1}^k \varepsilon_j \quad \sigma_{\varepsilon}^2 = \frac{1}{k} \sum_{j=1}^k (\varepsilon_j - \bar{\varepsilon})^2$$

El resultado es la **media** de los k errores, y la **varianza se reporta siempre con ella**: dice si el resultado es estable o si dependió de qué patrones tocaron. Una media de 8 % con desvío de 1 % y otra de 8 % con desvío de 6 % son resultados completamente distintos.

$$k = N \implies \text{leave-one-out}$$

El caso extremo: cada bloque es **un solo patrón**. Se entrena N veces y se mide sobre uno cada vez. Aprovecha el máximo de datos posible, pero cuesta N entrenamientos y la estimación tiene **mucho varianza**. Los valores usuales son $k = 5$ o $k = 10$, que equilibran costo y estabilidad.

Validación cruzada con $k = 5$: cinco entrenamientos, cada bloque es prueba una sola vez

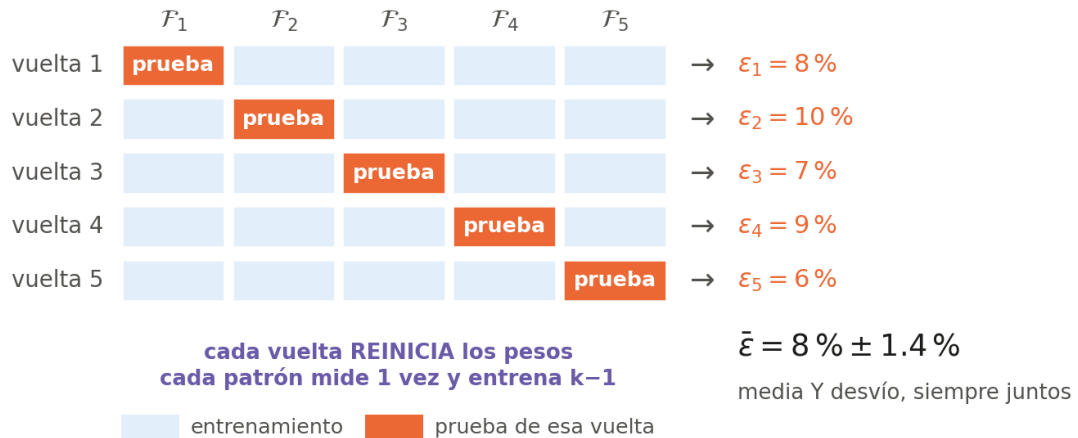


Figura 22: Con $k = 5$: cinco entrenamientos, y cada bloque hace de prueba exactamente una vez.

OJO — la validación cruzada evalúa un método, no entrega un modelo

Al terminar hay k redes distintas, y ninguna es «la» red. Lo que se estimó es qué tan bien anda **el procedimiento** (esta arquitectura, con estos hiperparámetros, sobre estos datos). Elegido el procedimiento, el modelo final se entrena **de nuevo, con todos los datos**. Y si dentro de la validación cruzada además hay que elegir hiperparámetros, el conjunto de monitoreo sale de los $k - 1$ bloques de entrenamiento — **nunca** del bloque que está haciendo de prueba.

Matriz de confusión

	predijo $\oplus$	predijo $\ominus$	total real
es $\oplus$	$t_{\oplus}$	$f_{\ominus}$	$N_{\oplus}$
es $\ominus$	$f_{\oplus}$	$t_{\ominus}$	$N_{\ominus}$

La tabla cruza **lo que era** (filas) contra **lo que dijo** (columnas). La diagonal son los aciertos; fuera de la diagonal, los dos tipos de error, que casi nunca cuestan lo mismo.

$$N_{\oplus} = t_{\oplus} + f_{\ominus} \quad N_{\ominus} = f_{\oplus} + t_{\ominus} \quad N = N_{\oplus} + N_{\ominus}$$

Los totales por fila son los que **realmente** pertenecen a cada clase. Ojo con la lectura de los nombres: $f_{\ominus}$ es «dijo negativo y erró», o sea que **era positivo**. El subíndice nombra lo que la red **dijo**, no lo que era.

Filas = lo que ERA · columnas = lo que DIJO · el subíndice de f nombra lo que DIJO ($f_{\ominus}$: dijo -, era +)

Sensibilidad (una FILA)

	dijo $\oplus$	dijo $\ominus$
era $\oplus$	$t_{\oplus}$	$f_{\ominus}$
era $\ominus$	$f_{\oplus}$	$t_{\ominus}$

$s^+ = t_{\oplus} / N_{\oplus}$
 de los que ERAN +, cuántos agarré

Especificidad (una FILA)

	dijo $\oplus$	dijo $\ominus$
era $\oplus$	$t_{\oplus}$	$f_{\ominus}$
era $\ominus$	$f_{\oplus}$	$t_{\ominus}$

$s^- = t_{\ominus} / N_{\ominus}$
 de los que ERAN -, cuántos dejé bien

Precisión (una COLUMNA)

	dijo $\oplus$	dijo $\ominus$
era $\oplus$	$t_{\oplus}$	$f_{\ominus}$
era $\ominus$	$f_{\oplus}$	$t_{\ominus}$

$p = t_{\oplus} / (t_{\oplus} + f_{\oplus})$
 de los que DIJE +, cuántos acerté

Exactitud (la diagonal)

	dijo $\oplus$	dijo $\ominus$
era $\oplus$	$t_{\oplus}$	$f_{\ominus}$
era $\ominus$	$f_{\oplus}$	$t_{\ominus}$

$a = (t_{\oplus} + t_{\ominus}) / N$
 la diagonal sobre el total

Figura 23: Cada medida mira una parte distinta de la tabla: una fila, una columna, o todo.

$$s^+ = \frac{t_{\oplus}}{N_{\oplus}} \quad s^- = \frac{t_{\ominus}}{N_{\ominus}} \quad p = \frac{t_{\oplus}}{t_{\oplus} + f_{\oplus}} \quad a = \frac{t_{\oplus} + t_{\ominus}}{N}$$

Sensibilidad s^+ : de los que **eran** positivos, cuántos agarré (una fila). **Especificidad** s^- : de los que **eran** negativos, cuántos dejé pasar bien (la otra fila). **Precisión** p : de los que **dije** positivos, cuántos acerté (una

Dice SIEMPRE «-»
95 negativos y 5 positivos

①

	dijo ⊕	dijo ⊖	
era ⊕	0	5	a 0,95
			s ⁺ 0,00
era ⊖	0	95	s ⁻ 1,00
			F ₁ 0,00

Un clasificador útil
95 negativos y 5 positivos

②

	dijo ⊕	dijo ⊖	
era ⊕	4	1	a 0,98
			s ⁺ 0,80
era ⊖	1	94	s ⁻ 0,99
			F ₁ 0,80

Exactitud casi igual (0,95 contra 0,98), utilidad opuesta: el primero no encuentra NINGÚN positivo. Nunca una sola medida.

Figura 24: Dos clasificadores con exactitud parecida y utilidad opuesta.

$$e = 1 - a \quad \delta_e = \frac{e_r - e}{e_r}$$

La mejora relativa respecto de un sistema de referencia: pasar de 98 % a 99 % de exactitud es **reducir el error a la mitad**, y pasar de 50 % a 51 % no es lo mismo aunque el salto de exactitud sea igual.

Medidas para predicción

$$e_i = y_i - \hat{y}_i \quad \tilde{\epsilon}_A = \frac{1}{N} \sum_i |e_i| \quad \tilde{\epsilon}_S = \frac{1}{N} \sum_i e_i^2 \quad \tilde{\epsilon}_R = \sqrt{\tilde{\epsilon}_S}$$

No se puede usar la suma de los errores: los signos se compensan. Por eso valor absoluto o cuadrado, y la raíz del cuadrático para volver a las **unidades del dato**. El cuadrático castiga más los errores grandes; el absoluto trata a todos igual.

Cómo se lee

Todo el protocolo se lee con **una sola pregunta por conjunto**: ¿este número participó de alguna decisión? El de entrenamiento participó de todas, así que no mide nada. El de monitoreo participó de una —cuándo parar y qué arquitectura—, así que mide de más. El de prueba no participó de ninguna: es el único que estima la generalización, y por eso se mira **una sola vez**.

Y la matriz de confusión se lee **por dirección**: sensibilidad y especificidad miran **una fila** cada una (de los que **eran** de esa clase, cuántos agarré); la precisión mira **una columna** (de los que **dije** de esa clase, cuántos acerté); la exactitud mira todo. Un clasificador que dice siempre lo mismo engaña a varias de ellas.

PARA LA DEFENSA — la respuesta de treinta segundos

«Generalizar es andar bien con datos que la red no vio, y como ese error no se puede calcular, se estima apartando datos desde el principio. Los datos se parten en tres conjuntos disjuntos y estratificados: entrenamiento ajusta los pesos, monitoreo decide cuándo parar y qué arquitectura usar, y prueba se mira una sola vez al final. Si hay pocos datos se usa validación cruzada de k particiones: se entrena k veces rotando cuál bloque hace de prueba, y se reporta la media de los k errores junto con la varianza.» **Las repreguntas seguras**: por qué no alcanza con dos conjuntos (porque el monitoreo decide, y lo que decide ya no puede medir); cómo se ve el sobreajuste (las dos curvas: entrenamiento baja, monitoreo toca un mínimo y sube —ahí se corta); por qué cada vuelta de la validación cruzada reinicializa los pesos; y por qué la exactitud sola no sirve (el clasificador trivial con clases desbalanceadas).

Los algoritmos, lado a lado

Algoritmo	Qué ajusta	Criterio	Competitivo	Parada
Perceptrón (corrección)	$\mathbf{w}$	$d - y$	no	sin errores en una pasada
Perceptrón (LMS)	$\mathbf{w}$	e^2	no	épocas / error
Multicapa	todas las $\mathbf{W}^{(p)}$	$\frac{1}{2} \sum e_j^2$	no	épocas / monitoreo
Base radial fase 1	μ_j	J de k -medias	sí	sin reasignaciones
Base radial fase 2	w_{kj}	$\frac{1}{2} \sum e_k^2$	no	épocas / error
Hopfield	w_{ji} (una vez)	— (Hebb)	no	N pasos sin cambios
BPTT	$\mathbf{W}, \mathbf{W}^I$	$\frac{1}{2} \sum_t \sum_k e^2$	no	épocas
SOM	$\mathbf{w}_j$ y vecinas	—	sí	épocas
LVQ1	$\mathbf{m}_c$	acierto de clase	sí	épocas

Las cuatro frases que ordenan todo

1. **Perceptrón, multicapa, RBF fase 2 y BPTT son el mismo esquema:** bajar por el gradiente de un error cuadrático. Lo que cambia es qué tan difícil es calcular ese gradiente.
2. **k -medias, SOM y LVQ son el mismo esquema:** gana el más cercano y se mueve hacia el dato. Lo que cambia es quién más se mueve — nadie, los vecinos de la grilla, o el ganador con signo.
3. **Hopfield es el único que no itera para entrenar:** una cuenta y listo. Y es el único que itera para usarse.
4. **La activación decide la forma de la frontera:** sigmoide, semiespacios; radial, burbujas locales.